

# GİRİŞ.

**Eğitmen**

**Özgür YILDIRIM**



**[linkedin.com/in/Ozgur-yldrm/](https://www.linkedin.com/in/Ozgur-yldrm/)**

**[github.com/OzgurYldrm](https://github.com/OzgurYldrm)**

**Sunum + Notebook + Homework → Github**

- **Text, Text Preproces**
- **Tokenization, Byte-Pair Encoding**
- **Word Vectors**
- **BoW, TFIDF**
- **CBOW, Skip-Gram, Negative Sampling**
- **Word2Vec, GloVe, FastText**
- **N-Gram Models**
- **Greedy Search, Beam Search**
- **RNN, LSTM, GRU**
- **Attention, Self-Attention**
- **Transformer**

## Text

**Text → yazı verisidir.**

**İnternet üzerinden “web scraping” yöntemi ile toplanan veriler, şirket dökümanları, el yazısının dönüştürülmüş versiyonu vb. birçok farklı türü vardır.**

**Corpus:** Bir metnin/dökümanın tamamını ifade eder. Örneğin yapay zeka modeli eğitmek için sosyal medya platformu üzerinden topladığımız 1000 adet yorum bizim model eğitim için kullandığımız corpus olur.

**Text verisi kendi başına Large Language Model (LLM) modeli eğitmek için kullanılabilir. LLM’ler günümüzde birçok farklı alanda (image, tabular, ses, zaman serisi vb.) kullanılabilir (multimodal) . Bu module’ün amacı sadece ham yazı verisine odaklanmak.**

## Text Preprocess

**Lowercasing:** Tüm harfleri küçük harfe çevirme işlemi. Büyük ve küçük harfle başlayan aynı kelime bilgisayar tarafından farklı olarak algılanır. Bu yüzden lowercasing uygulanabilir.

*“Elma” → “elma”*

**Punctuation Removal:** Noktalama işaretlerinin kaldırılmasıdır.

*“I love NLP!” → “I love NLP”*

**Stopword Removal:** Stopword kelimelerin kaldırılmasıdır.

*"Hello there! This is an example, showing basic text preprocessing."*

*['hello', 'example', 'showing', 'basic', 'text', 'preprocessing']*

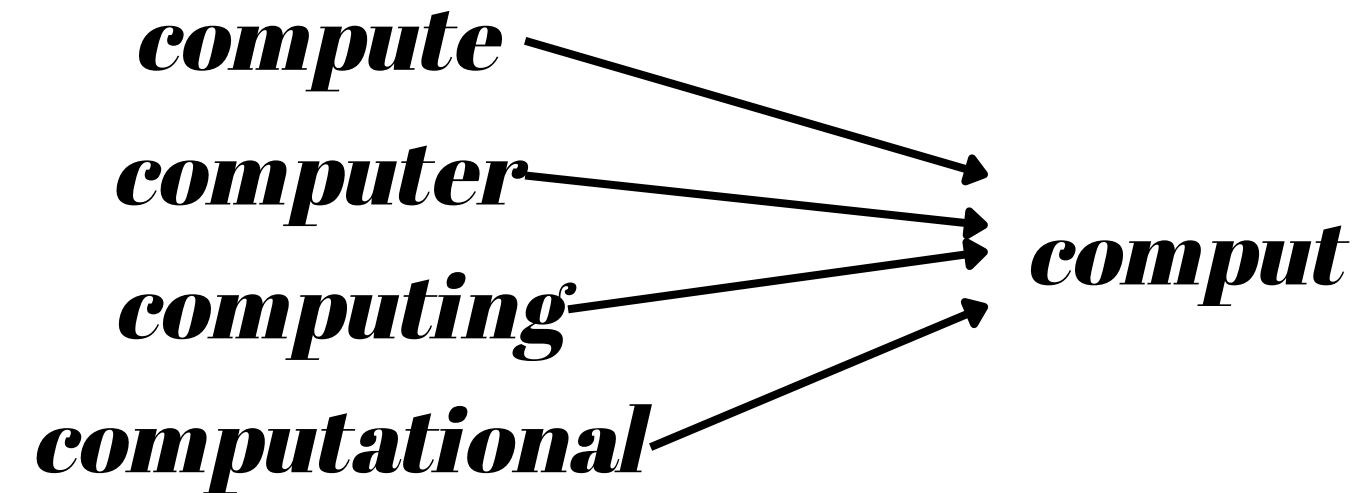
**Digit Removal:** Sayıların kaldırılmasıdır.

*“3 elma” → “elma”*

Bu işlemler uygulanırken dikkat edilecek nokta uygulanan işlemin problemi nasıl etkileyeceğidir. Örneğin yazım denetleme yazılımının içerisinde lowercasing sonucu olumsuz etkiler ancak duygu tespiti içerisinde digit removal işlem yükünü azaltabilir.

# Stemming

**Stemming:** Kelimelerin köklerini bulma işi



**Stemming kelimeleri normalize etmek için güzel bir yöntemdir ancak dil çok çeşitli bir yapı olduğu için genelde yanlış köklere ulaşılabilir.**

*“studies” -> “studi” (crude stem)*  
*dolabın → dolab*

***Lovin’s Stemmer: Önceden belirlenmiş ekleri çıkarır.***

***Porter Stemmer: 5 kural çerçevesinde ekleri çıkarır.***

## Lemmatization

**Kelimelerin kökünü bulmayı amaçlar. Stemming'den farkı kök bulmaya çalışırken ekleri ayırmaz, dictionary kullanır.**

**{**  
**...**  
***better: good***      **“*better*” → “*good*”**  
**...**      **“*ran*” → “*run*”**  
***ran: run***  
**}**

**Lemmatization için kelimelerin bulunduğu bir dictionary bulunması gerekir. Ayrıca daha önce görülmemiş bir kelime geldiğinde bunu köküne ayıramayabilir.**



## Stemming-Lemmatization

**Stemming-Lemmatization gerekli işlem yükünü azaltabilir ancak zaman kipleri vb. grammar yapılarının kaybolmasına sebep olabilir.**

***Bu makine öğrenmesi dersi çok faydalı.  
bu makine öğren ders faydalı*** } ***Eğer yorum analizi yapan bir algoritma  
geliştiriliyorsa bu işlem sıkıntı  
çıkarmayabilir***

***Bu filmi sevdim  
Bu filmi sevmedim*** } ***Bu film sev*** } ***Eğer yorum analizi yapan bir algoritma  
geliştiriliyorsa bu işlem sıkıntı  
çıkabilir.***

# Tokenization

**Tokenization:** Bir metni daha küçük anlamlı parçalara ayırma işlemidir. Elde edilen her bir küçük parçaya **token** denir. Token dil modellerinin eğitiminde kullanılan en küçük parçalardır.

**Tokenization ile arayış:** Kendi içerisinde anlamı olan en küçük parçaları elde etmektir.

**İlerleyen kısımlarda her bir token'a anlamsal bir sayı değerinin nasıl atanabileceğini göreceğiz.**

## Letter (Byte-Character) Level Tokenization

- Metin her bir karakter baz alınarak ayrılır. Kullanılan alfabede kaç tane harf varsa o kadar unique token var demektir.

*"May the Force be with you"*

*['M','a','y',' ','t','h','e',' ','F','o','r','c','e',' ','b','e',' ','w','i','t','h',' ','y','o','u']*

- Letter-based tokenization her bir token içerisinde yeterli anlamı sağlayabilecek bir yöntem değildir.

*apay zeka*  
*Y azılım*  
*OLO*



- Her harf için bir token atamak “*computationally expensive*” dir. Ortalama bir metinde 1000 harf geçiyorsa her bir harf için memory’de yer açmak gerekir.
- Spell-checking gibi harflerin önemli olduğu projeler için bu yöntem kullanılabilir.

*Yolda yavru bir köpke gördüm.*



## Word Level Tokenization

- Metin her bir kelime baz alınarak ayrılır. Corpus içerisindeki unique kelime sayısı kadar unique token olur.

*"May the Force be with you ."*

*['May', 'the', 'Force', 'be', 'with', 'you', '.']*

- Corpus içerisinde olmayan bir kelime ile karşılaşılsa temsil edilemez. Yazım hatası gibi bir durum buna sebep olabilir.

**Train Corpus**

*Yavru kedi bahçede oynuyor.*

**Test Corpus**

*kedy çok tatlı*

*kedy* → *<UNK>*

*<UNK>* → **Unknown token**

## Sentence Tokenization

- **Metin her bir cümle baz alınarak ayrılır. Bu noktada arayış her cümleyi unique bir token'la temsil etmek değil, metinde cümleler arasındaki bağlantıyı yakalamaktır.**

*"May the Force be with you. Remember, the Force will be around you always."*

*['May the Force be with you.', 'Remember, the Force will be around you always.']*

- **Genellikle duygu analizi, özetleme, bilgi çıkarımı vb. konularda kullanılır.**

## Sub-Word Tokenization

**Subword Tokenization:** Kelimelerin köklerini ve eklerini token olarak değerlendirir. Teoride her bir kök ve ek kendi içerisinde tutarlı bir anlama sahip olduğu için en sık kullanılan yöntemdir.

*"May the Force be with you."*

*["May", "the", "For", "ce", "be", "with", "you", "."]*

- Subword tokenlarının nasıl belirleneceği önemli bir kısımdır. Token'ları belirlemek için kullanılan farklı algoritmalar vardır. Farklı subword tokenizer'lar farklı şekilde ayırabilir.
- Güncel “Tokenizer”lar bu metoda benzer yapılar kullanır. Eğer train corpus içerisinde olan bir kelime geldiyse doğrudan kelimeyi bir token gibi sayma eğilimleri vardır.

*"May the Force be with you"*

*['May', 'the', 'Force', 'be', 'with', 'you', '.']*

- Subword tokenization ile <UNK> problemi genellikle ortadan kalkar. Daha önce görülmemiş bir kelime geldiğinde harflere/hecelere ayrılarak ifade edilebilir.

# <sup>1</sup>Byte-Pair Encoding

**Subword tokenization öğrenme algoritmasıdır. En sık tekrar eden kalıpları token olarak belirler.**

**“*work*” → “*worker*”      *##er* → token**

**BPE iteratif adımda sadece bir adet yeni token öğrenir.**

---

**Corpus: “A B D C A B E C A B”**

**1. Corpus içerisindeki bütün unique karakterleri token listesine ekle.**

**Tokens = { A,B,C,D,E }**

**2. En sık tekrar eden kuralı bul. Token listesine ekle.**

**En sık tekrar → “AB”      Tokens = { A,B,C,D,E,AB }      “AB” öğrenildi**

**Corpus: “AB D C AB E C AB”**

**3. En sık tekrar eden kuralı bul. Token listesine ekle.**

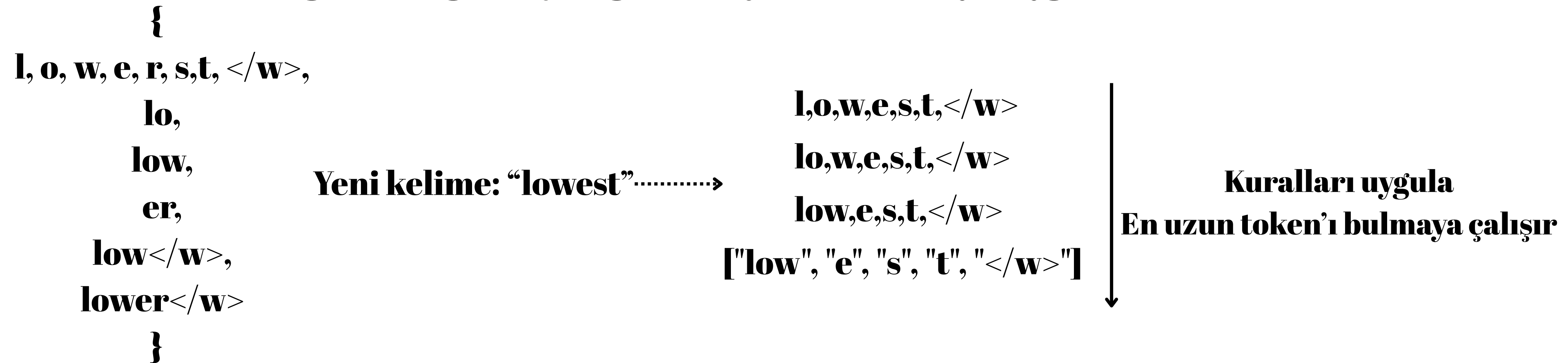
**En sık tekrar → “CAB”      Tokens = { A,B,C,D,E,AB ,CAB }      “CAB” öğrenildi**

**Corpus: “AB D CAB E CAB”**

## <sup>2</sup>Byte-Pair Encoding

- Başlangıçta bütün kelimeler karakterlerine ayrılır.
- Pratikte kelime sonuna `</w>` token'ı koyulabilir.
- Tüm corpus içinde frekans sayımı yapılır. En sık geçen token oluşturulur.
- Token listesi güncellenir. Corpus yeni token'lara göre şekillendirilir.
- Eğer eşit frekansa sahip diziler varsa deterministik bir yöntem uygulanır: Soldakini al, alfabe de önce geleni al vb.
- Belirli bir vocab (V) sayısına ulaşıldığında veya frekans belli bir değerin altına düştüğünde durulur.

**Tokenization:** Yeni gelen bir girdi için öğrenilmiş kurallar sırayla uygulanır.





# Word Vectors

**Yapay Zeka modelleri text verisini doğrudan anlayamaz → Sayıya/vektöre dönüştür.**

**Kelimeleri vektör olarak ifade etmek için farklı yöntemler vardır. Hepsinin belirli avantajları ve dezavantajları olabiliyor.**

| <b>İstatistik Tabanlı Modeller</b> | <b>Derin Öğrenme Tabanlı Modeller</b> |
|------------------------------------|---------------------------------------|
| <b>Bag of Words</b>                | <b>CBOW</b>                           |
| <b>TF/IDF</b>                      | <b>Skip-Gram</b>                      |
|                                    |                                       |

# One-Hot Word Vectors

**En basit vektör oluşturma yöntemidir. Her bir unique kelime için One-Hot vektör oluşturulur.**

$$\text{kedi} = \begin{bmatrix} 0 \\ 0 \\ \dots \\ 1 \\ 0 \\ \dots \\ 0 \end{bmatrix}$$

**Sadece tek bir girdisi (sıra) 1'dir geri kalanlar 0'dır → sparse**  
**Semantic anlam yoktur.**

**Vektör boyutu Vocab boyutudur. Bu yüzden aşırı büyüktür.**  
**Örnek olarak 200.000 kelimelik bir sözlük Vocab olarak**  
**kullanılırsa vektör boyutu 200.000 olur.**

**Çoğu zaman doğrudan bu yöntem token'ları vektöre**  
**dönüştürmek için kullanılmaz. Ancak çoğu yöntemin**  
**başlangıç noktasıdır.**

**Cümledeki kelimelerin sırası vektörü etkilemez.**

|                                                                                               |                                                                            |
|-----------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| $\left. \begin{array}{l} \textit{cat bite dog} \\ \textit{dog bite cat} \end{array} \right\}$ | <b>cat kelimesi için oluşturulan</b><br><b>vektör her ikisinde de aynı</b> |
|-----------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|

# Dense Word Vectors

$$\text{kedi} = \begin{bmatrix} 0.286 \\ 0.792 \\ -0.177 \\ \dots \\ 0.349 \\ 0.271 \\ -0.002 \end{bmatrix}$$

**Her bir dimension kendi içinde bir özelliği barındırır.**

**Dense vector'ün benzerliği ölçülebilir.**

**Vector dimension developer seçimidir. (Hyperparameter)**

**Model türüne göre → Kelime sırası önemli olabilir.**

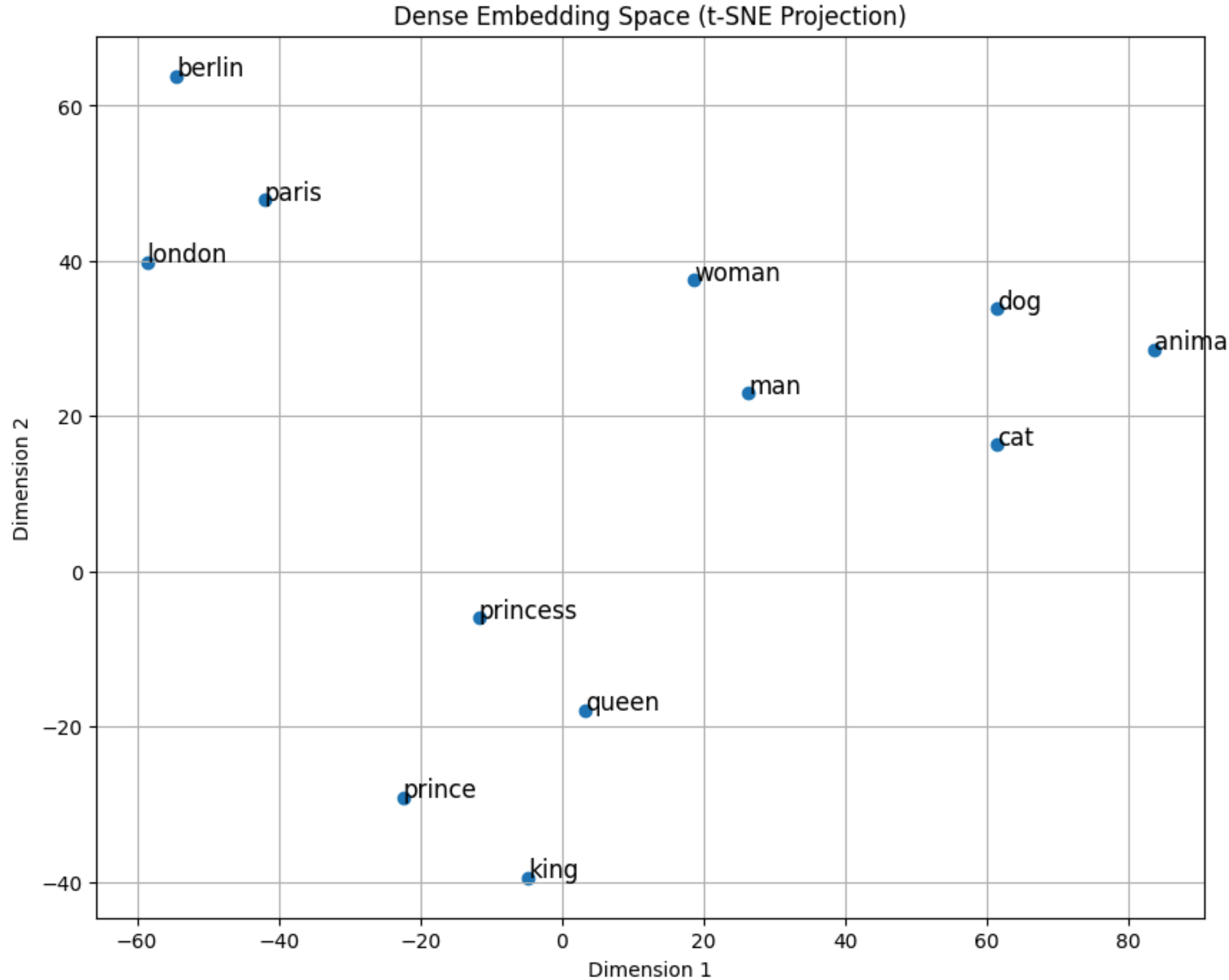
*cat bite dog*  
*dog bite cat*

}

**cat kelimesi için oluşturulan  
vektör her ikisinde de aynı  
olmayabilir**



# Dense Word Vectors



**Semantic yakalayabilir. Farklı bağlamlarda birbirine benzeyen kelimelerin girdileri yakın olur. Dolayısıyla uzayda benzer konumlarda olurlar.**

# <sup>1</sup>Vector Similarity

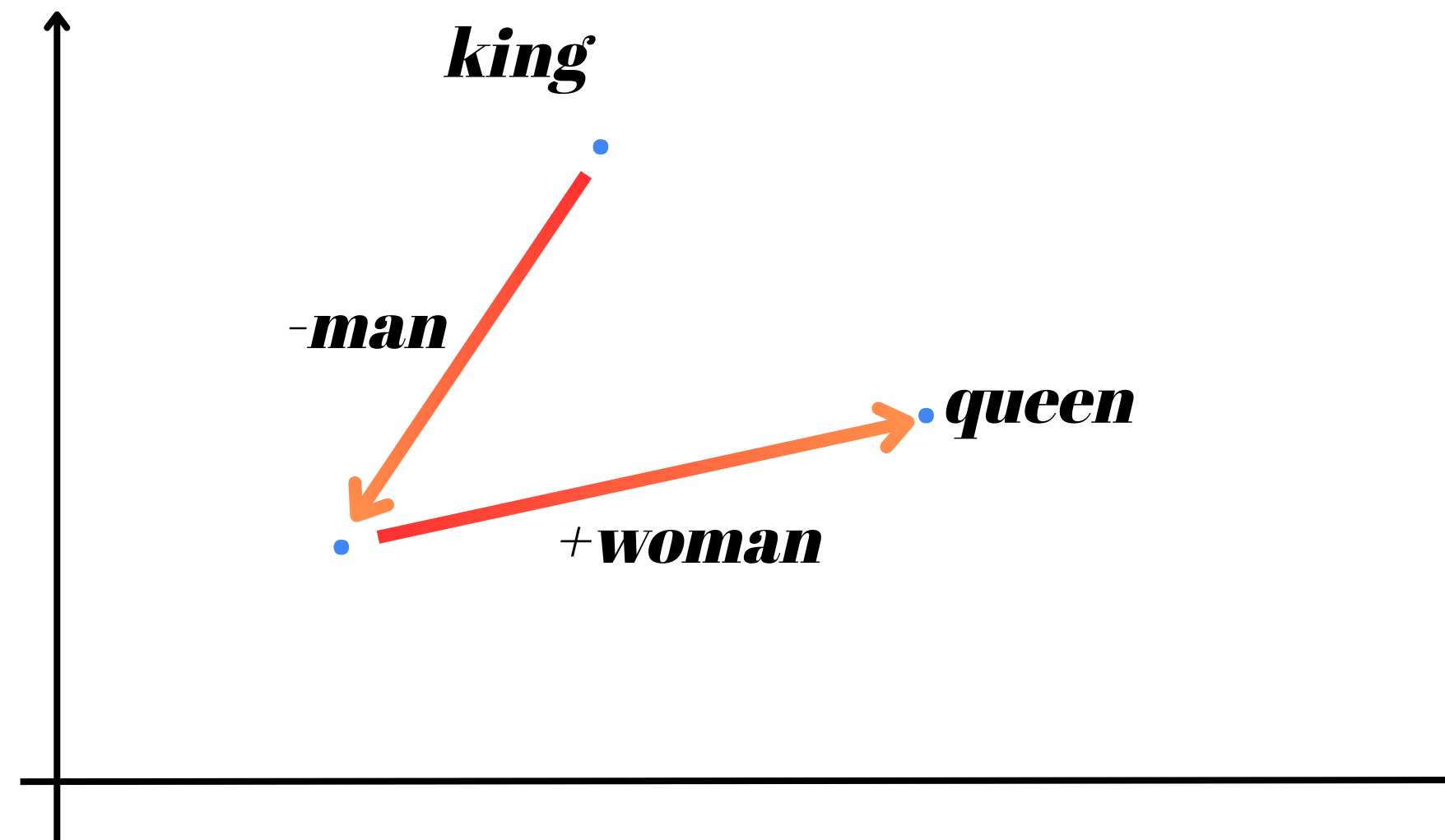
İki vektörün benzerliğini **Dot Product** olarak hesaplayabiliriz.

*“apple” . “chair”* → 3.4636424

*“apple” . “banana”* → 16.466734

Vektörler üzerinde toplama çıkarma işlemleri semantik anlamı bozmaz.

*king* − *man* + *woman* ≈ *queen*

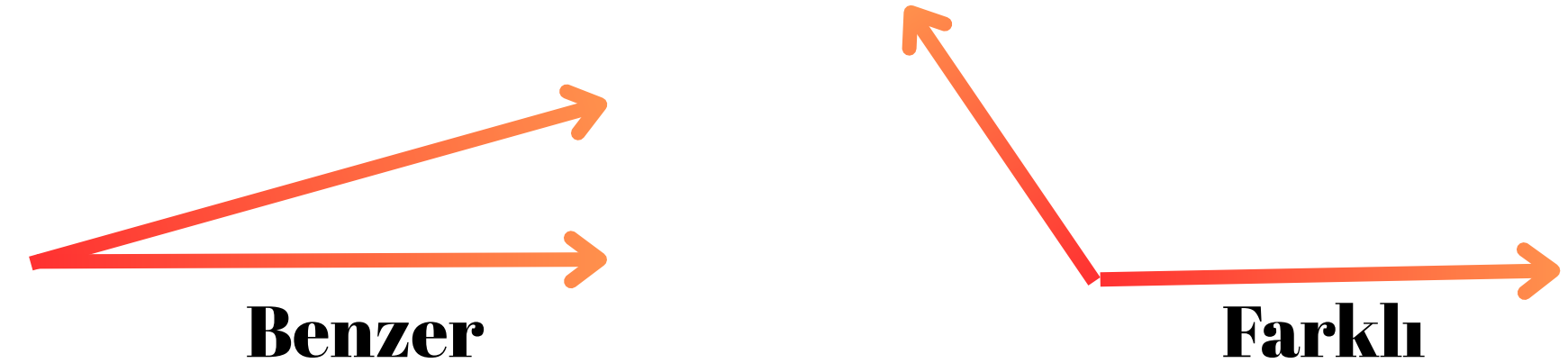
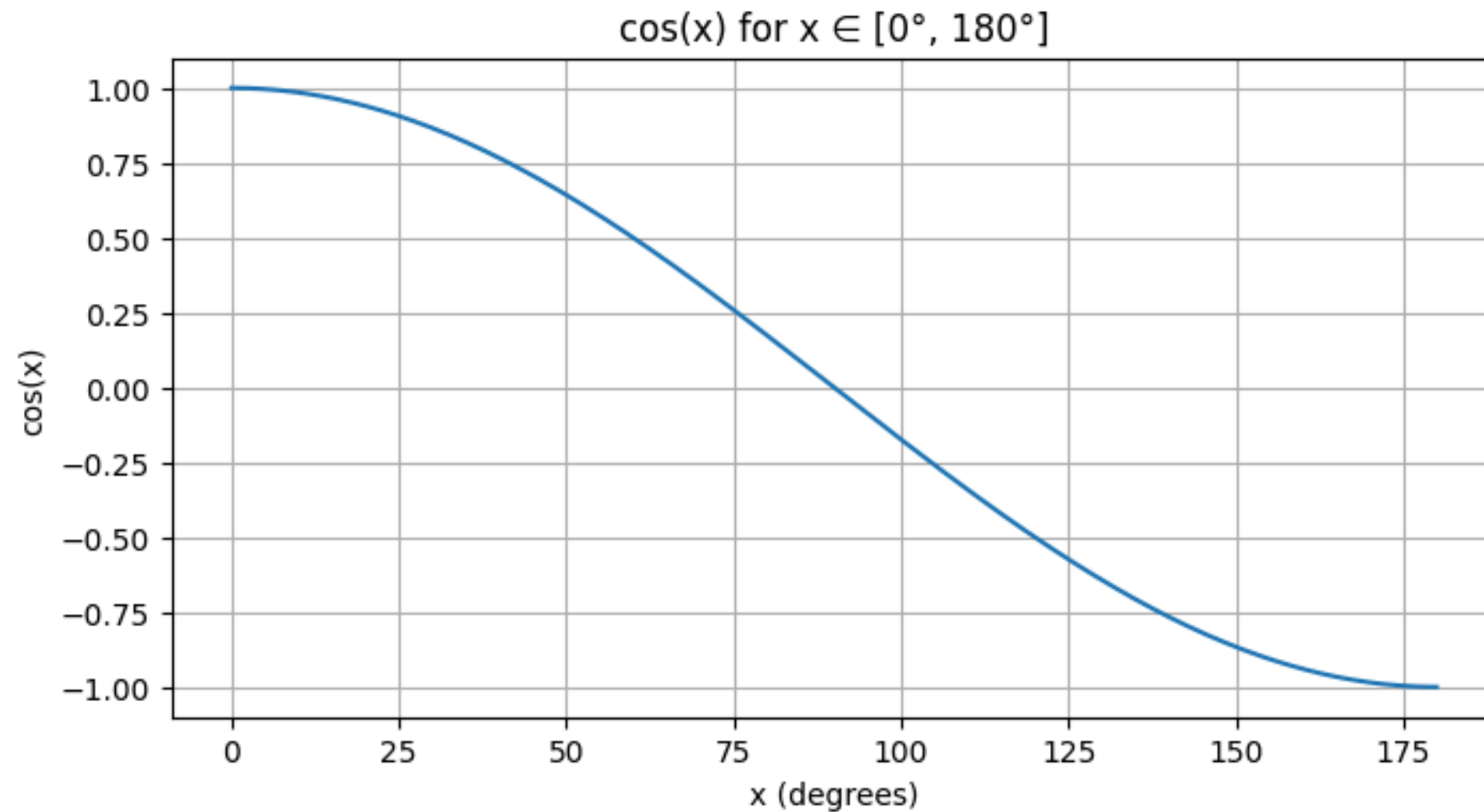


## <sup>2</sup>Vector Similarity

$$\cos(\theta) = \frac{\vec{A} \cdot \vec{B}}{\|\vec{A}\| \|\vec{B}\|}$$

Dot product büyüklüğü de ölçtüğü için uzun vektörler daha benzer çıkar. **Cosine Similarity** normalize edilmiş versiyonudur.

Benzer vektörler arasındaki açı küçüktür, açının cos değeri de küçüktür.



Eğer vektörler normalize edilmiş ise (uzunlukları 1 ise) paydayı hesaplamaya gerek yoktur (Dot product).

# Bag of Words



Her bir kelimenin ne kadar geçtiğini sayar



- Basit, anlaşılır, interpretable
- Kelime sırası ve bağlamı kaybolur. Semantic yok.

*Dog bit man - Man bit dog* Aynı vektör

- Sparse Vector üretir. (SVM modeli bu konuda avantajlı olabilir.)

*kedi oturdu*  $\rightarrow$  d: 200000

# Term Frequency (TF)

Bir kelimenin dökümanda ne sıklıkla geçtiğini ölçer

$$TF(t,d) = \frac{\# \text{ t appears in d}}{\# \text{ total terms in d}}$$

Document 1 → “The cat sat on the mat”

Document 2 → “The dog played in the park”

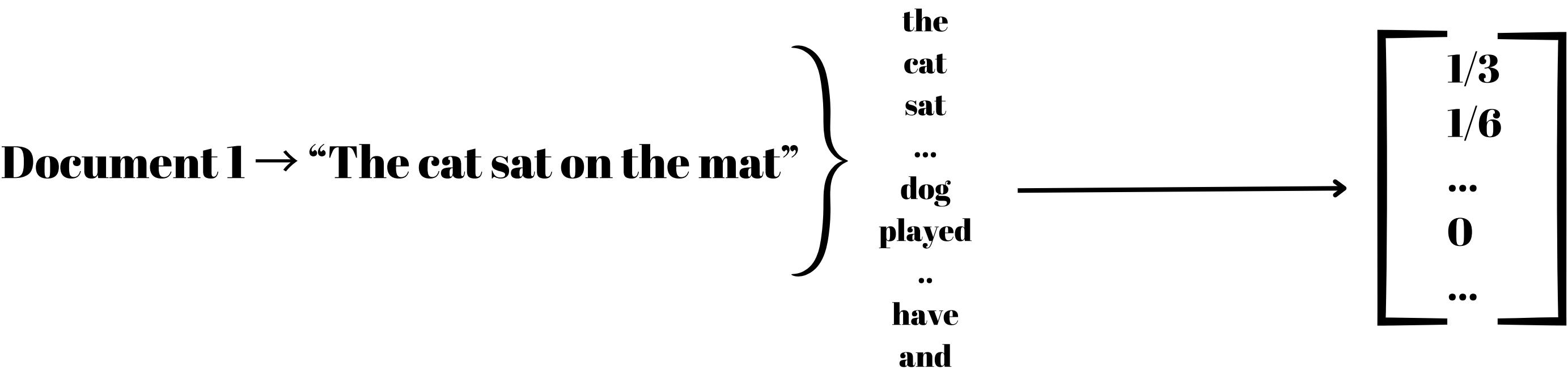
Document 3 → “We have the cat and the dog”

TF(“cat”,1) = 1/6

TF(“cat”,2) = 0

TF(“cat”,3) = 1/7

Vector olarak ifade etmek için vocab içindeki bütün kelimeler için TF hesaplarız.





# Inverse Document Frequency (IDF)

**Sık geçen gramatik kelimelerin etkisini azaltır. Örneğin “*the*” kelimesi her metinde sıklıkla geçer. Model eğitimi sırasında bias yaratabilir.**

$$\text{IDF}(t,D) = \log \left( \frac{\# \text{ total document in } D}{\# \text{ document containing } t} \right) \quad \begin{array}{l} * \log \rightarrow \ln \\ D: \text{ bütün dökümanlar} \end{array}$$

**Document 1 → “*The cat sat on the mat*”**  
**Document 2 → “*The dog played in the park*”**  
**Document 3 → “*We have the cat and the dog*”**

**IDF(“cat”,D) = log(3/2)**

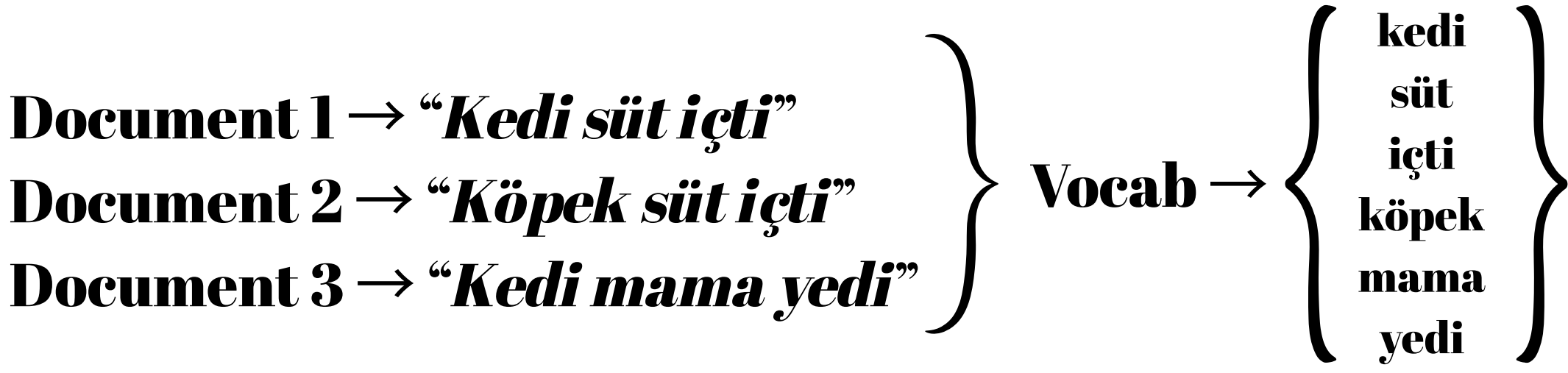
**IDF tüm corpusa ait istatistiksel bir özelliktir. Tek bir dökümana ait bilgi çıkarmaz.**

**Her document içinde geçen kelimelerin IDF’i 0 çıkar. Bu da “*The*”, “*a*” gibi kelimelerin etkisini azaltır.**

# TF-IDF

**TF ve IDF’i tek formülde birleştirir. Sonuçta kelime frekansı bazlı bir vektör verir.**

$$TD-IDF(t,d,D) = TF(t,d) \times IDF(t,D)$$



|                                              |             |                                                                                              |              |                                                                                                             |                |                                                                                                    |
|----------------------------------------------|-------------|----------------------------------------------------------------------------------------------|--------------|-------------------------------------------------------------------------------------------------------------|----------------|----------------------------------------------------------------------------------------------------|
| <b>Document 1</b> → “ <b>Kedi süt içti</b> ” | <b>TF =</b> | <div><div>0.33</div><div>0.33</div><div>0.33</div><div>0</div><div>0</div><div>0</div></div> | <b>IDF =</b> | <div><div>0.405</div><div>0.405</div><div>0.405</div><div>1.099</div><div>1.099</div><div>1.099</div></div> | <b>TFIDF =</b> | <div><div>0.1347</div><div>0.1347</div><div>0.1347</div><div>0</div><div>0</div><div>0</div></div> |
|----------------------------------------------|-------------|----------------------------------------------------------------------------------------------|--------------|-------------------------------------------------------------------------------------------------------------|----------------|----------------------------------------------------------------------------------------------------|

\*IDF vektörü  
bütün  
dokümanlar  
için aynı

# <sup>1</sup>Continuous Bag of Words (CBOW)

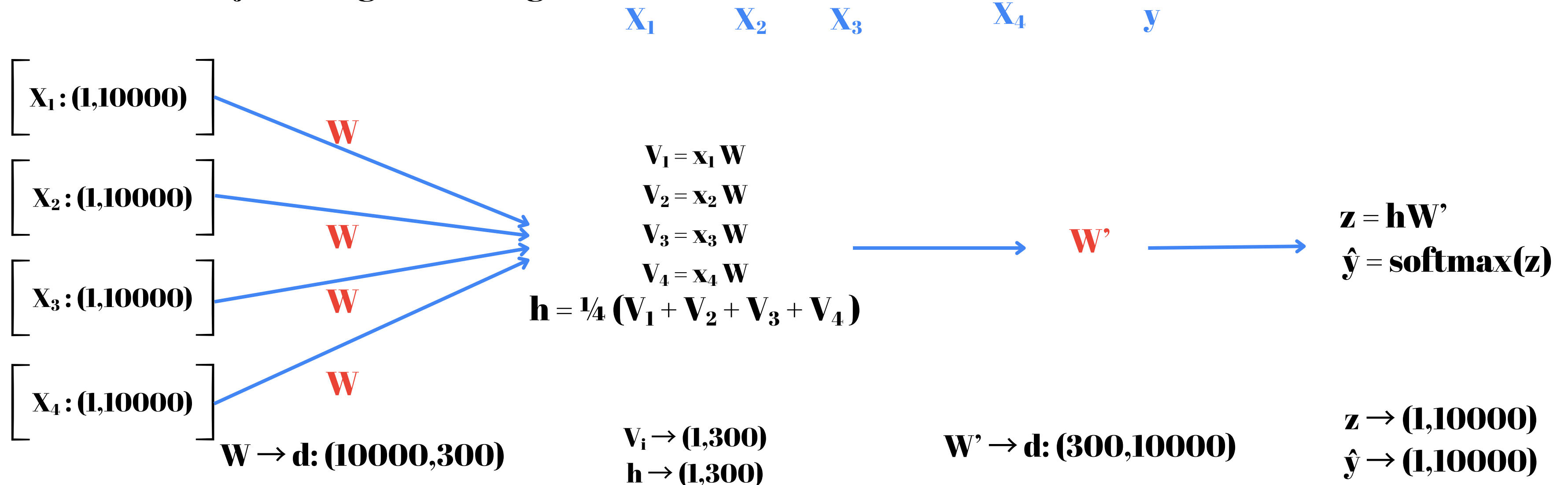
**Distributional semantics:** You shall know a word by the company it keeps. (J. R. Firth)

Kelime  $\rightarrow$  Embedding Vector | Context word kullanarak center word tahmini



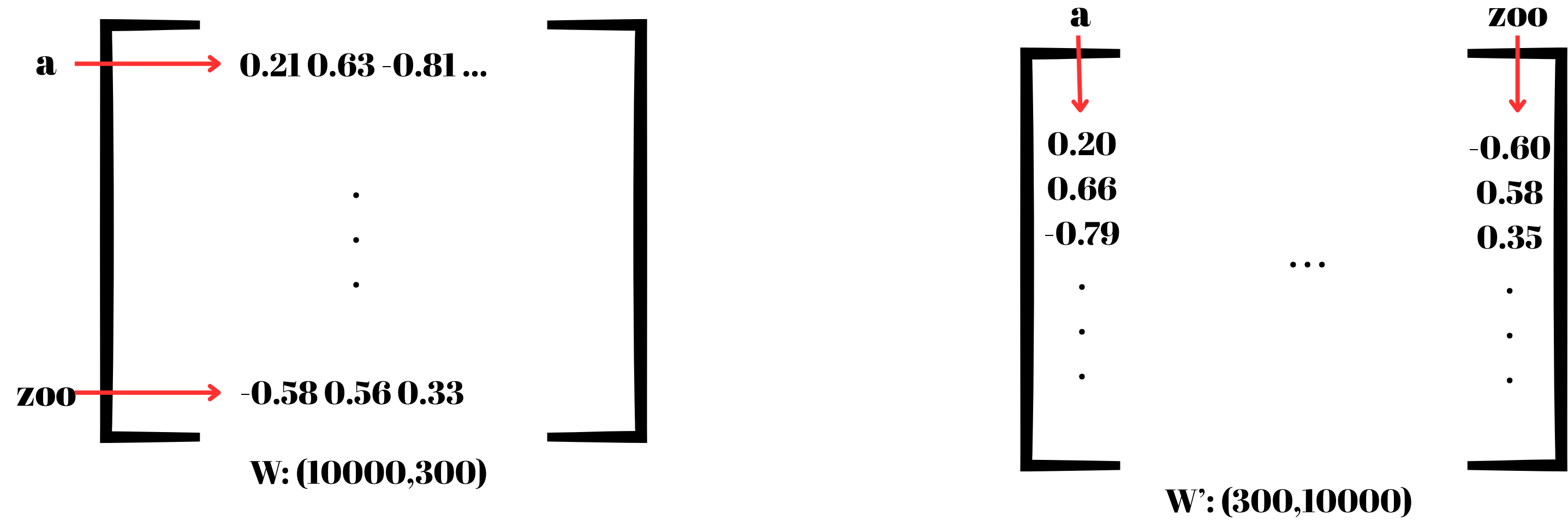
CBOW modeli için bir eğitim örneği : ( (“not”, “who”, “am”, “underneath”), “I”)

$X \rightarrow d: (1, 10000)$

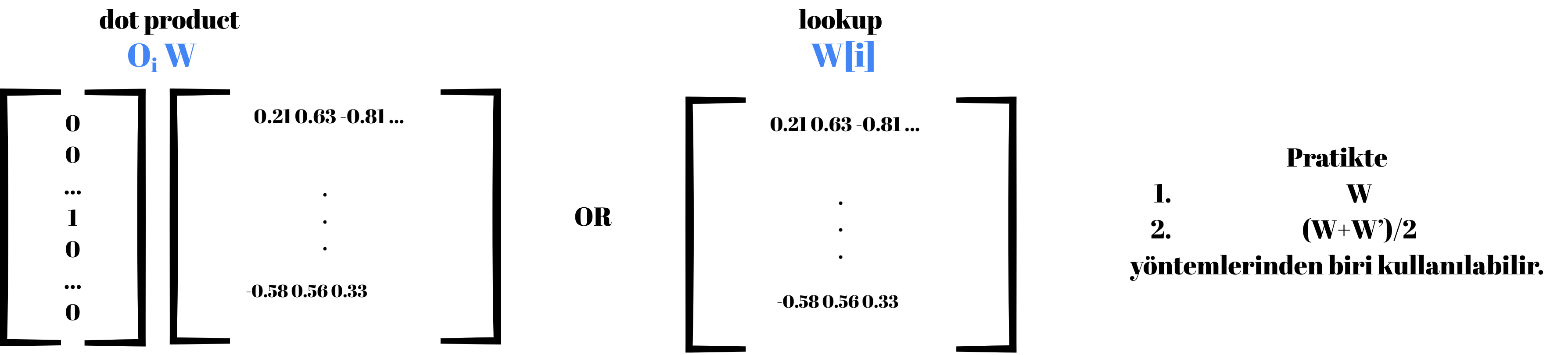




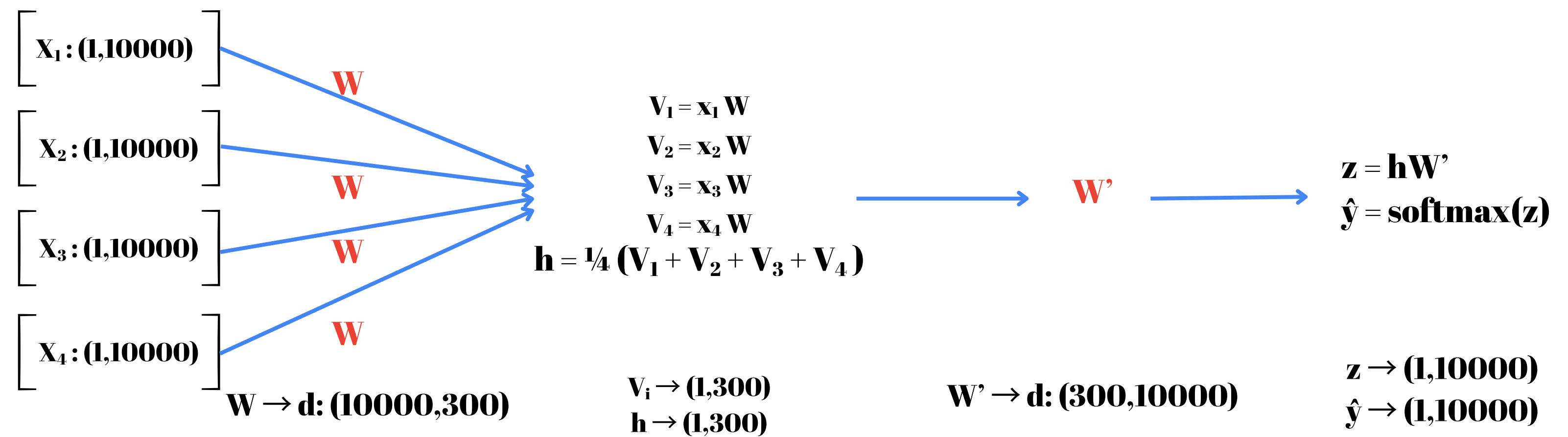
## <sup>2</sup>Continuous Bag of Words (CBOW)



Model eğitimi tamamlandıktan sonra Vocab içinde bulunan bir kelimenin (i) embedding vektörünü elde etmek için:



### Continuous Bag of Words (CBOW)



Veriyi

$\mathbf{x}_1 \mathbf{W} + \mathbf{x}_2 \mathbf{W} + \mathbf{x}_3 \mathbf{W} + \mathbf{x}_4 \mathbf{W} = (\mathbf{x}_1 + \mathbf{x}_2 + \mathbf{x}_3 + \mathbf{x}_4) \mathbf{W} \longrightarrow ((\mathbf{x}_1 + \mathbf{x}_2 + \mathbf{x}_3 + \mathbf{x}_4), y)$

formatında saklayabiliriz

# <sup>1</sup>Skip-Gram

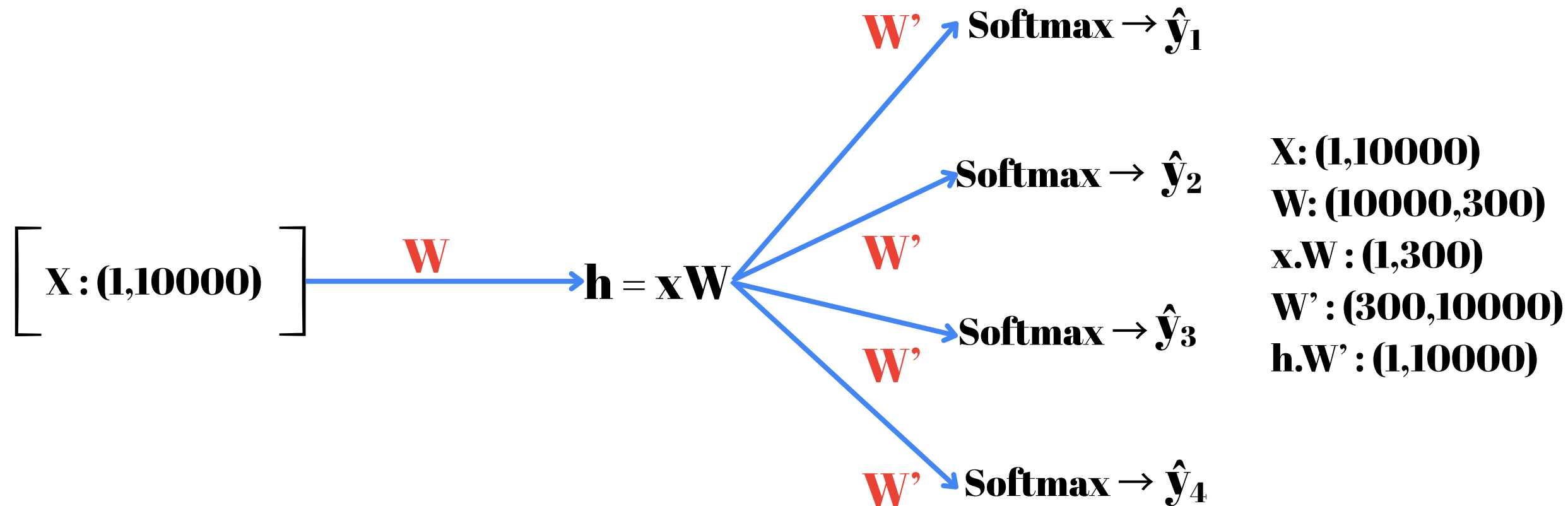
Kelime  $\rightarrow$  Embedding Vector | Center Word kullanılarak Context word tahmini



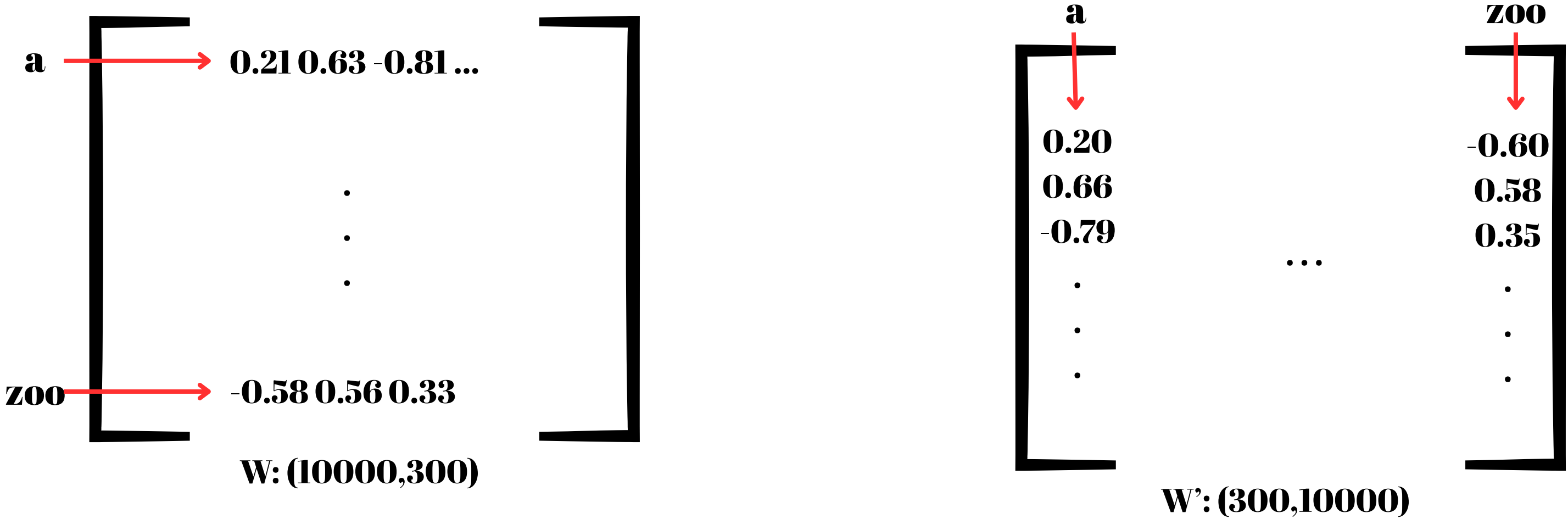
Skip-Gram modeli için bir eğitim örneği : ( "I", ("not", "who", "am", "underneath"))

$X \rightarrow d: (1, 10000)$

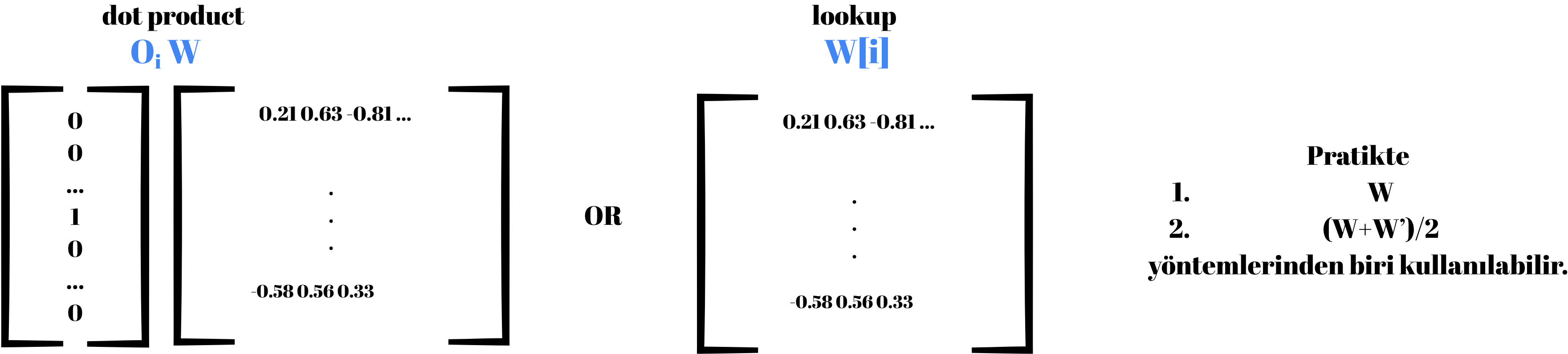
$X$     $y_1$     $y_2$     $y_3$     $y_4$



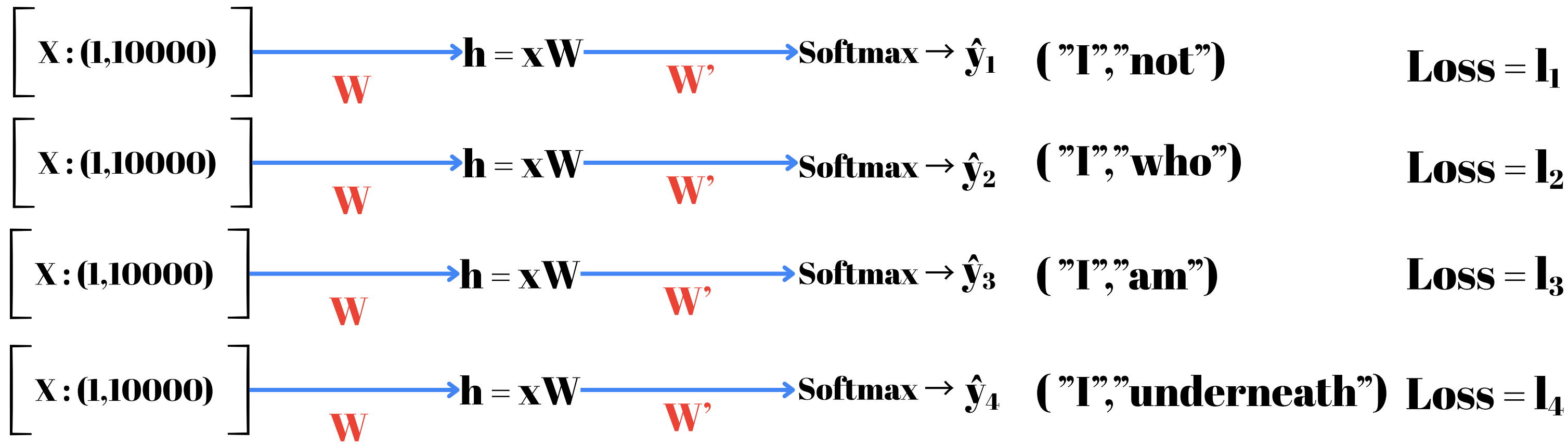
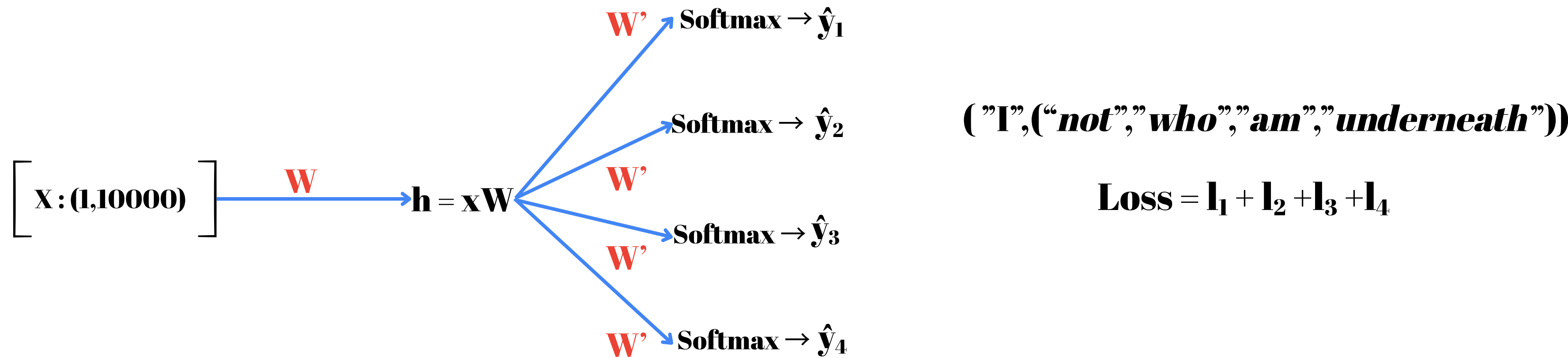
# 2Skip-Gram



Model eğitimi tamamlandıktan sonra Vocab içinde bulunan bir kelimenin (i) embedding vektörünü elde etmek için:



### ³Skip-Gram





# <sup>1</sup>Negative Sampling

$$P(w_t = i \mid h) = \frac{\exp(u_i^\top h + b'_i)}{\sum_{j=1}^{10000} \exp(u_j^\top h + b'_j)} \longrightarrow 10000 \text{ kez tekrar ediyor.}$$

**CBOW veya skip-gram modelleri aynı anda V=10000 kelime için olasılık hesaplar ve modeli eğitir. Negative sampling pozitif 1 adet kelime ve negatif k adet kelime için modeli eğitir.**

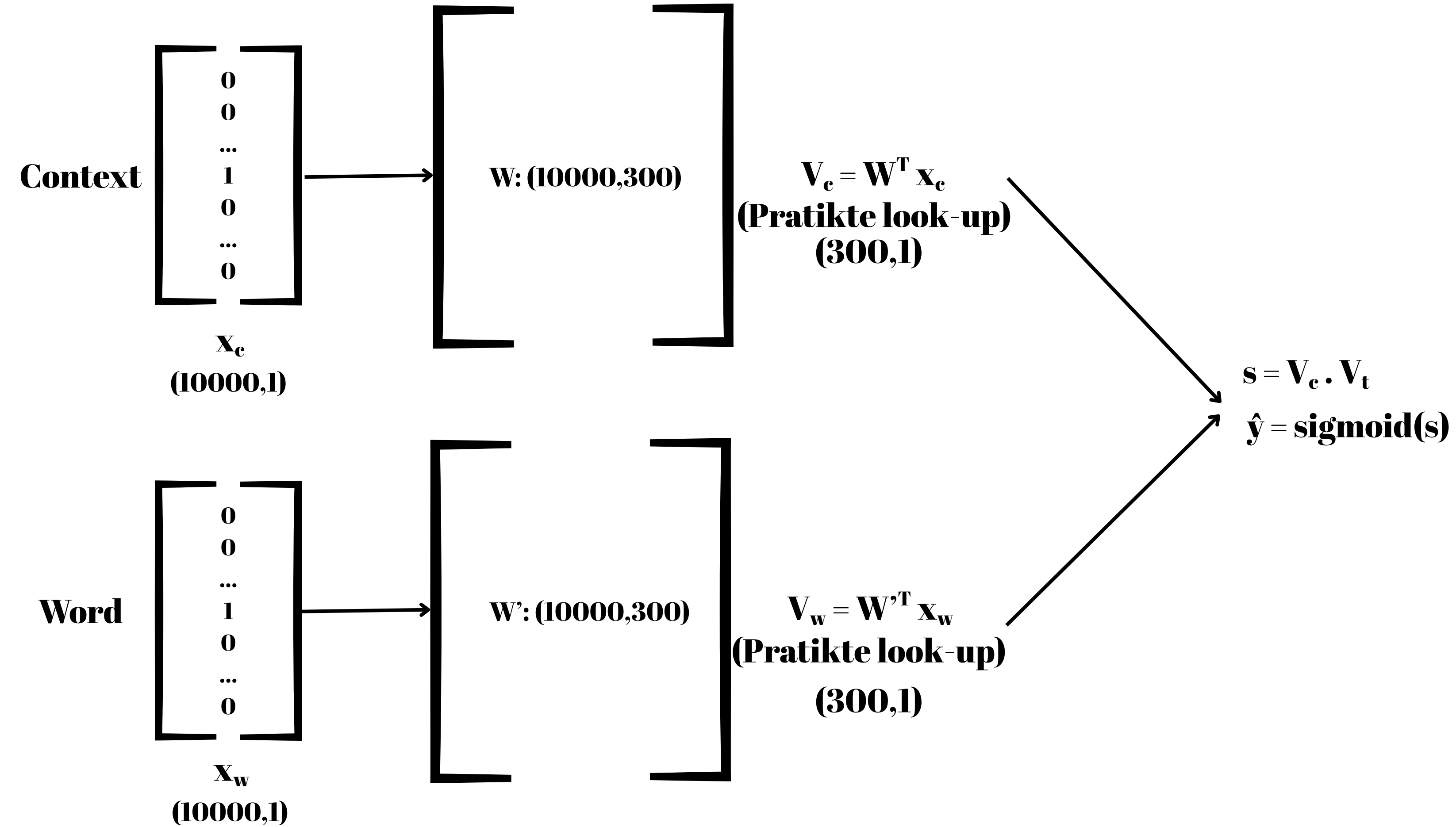
*The first rule of Fight Club is; you do not talk about Fight Club.*

| Context | Word | target |
|---------|------|--------|
| fight   | club | 1      |
| fight   | book | 0      |
| fight   | car  | 0      |

(“fight”, “club”, 1)  
 (“fight”, “book”, 0)  
 (“fight”, “car”, 0)

**K**  $\longrightarrow$  V içinden rastgele K kadar kelime seç  
 Küçük veri seti için: 5-20  
 Büyük veri seti için: 2-5

## <sup>2</sup>Negative Sampling



# <sup>1</sup>Word2Vec

**\*Tomas Mikolov, Kai Chen, Greg Corrado, Jeffrey Dean. 2013. Efficient Estimation of Word Representations in Vector Space**

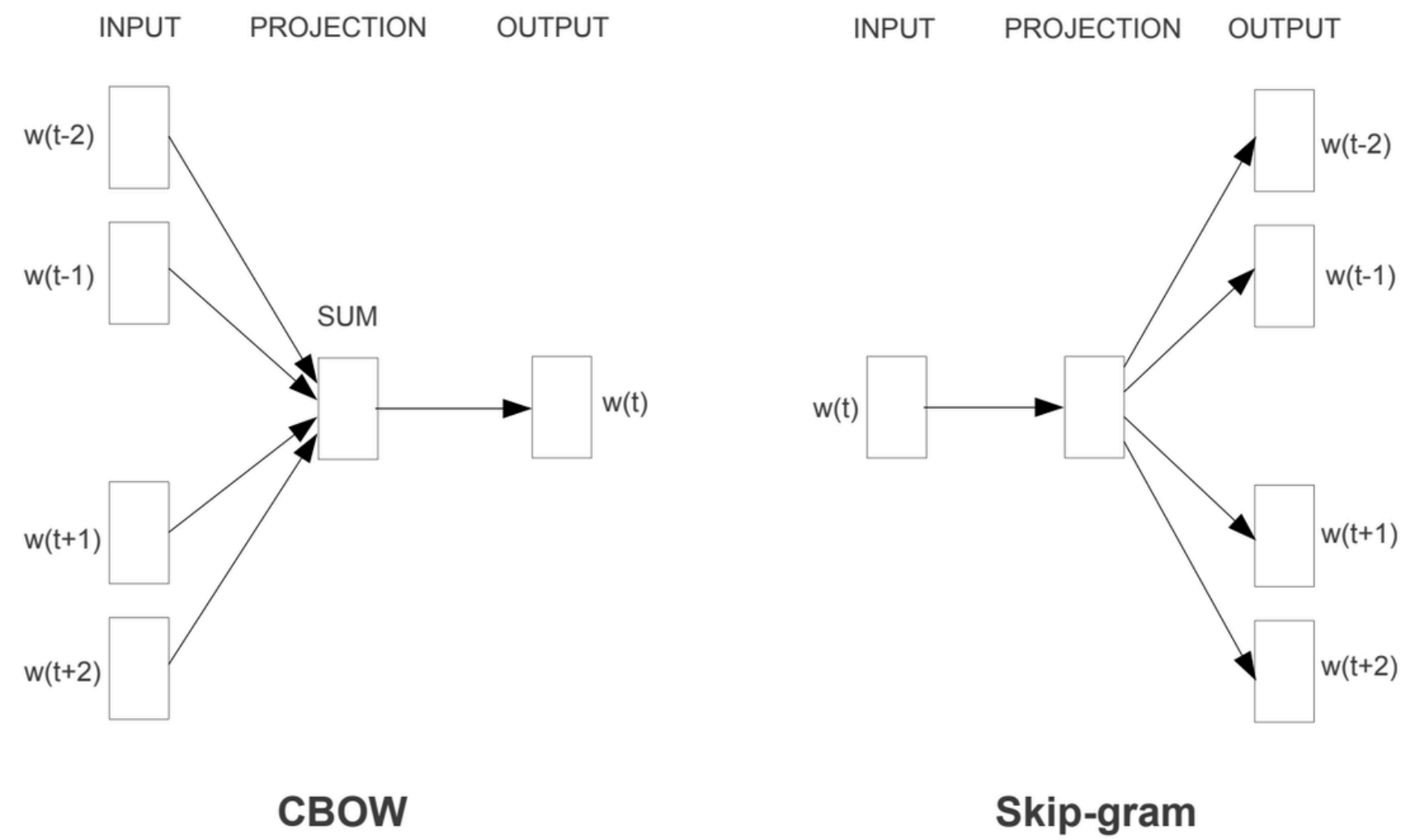
**One-Hot Vector → Continuous Vector**

**NNLM → Çok benzer. Bir hidden layer fazla**  
**RNNLM → Hesaplama ve optimizasyon problemleri**

**Complexity:**

$O = E \times T \times Q$

- E: epoch**
- T: # words**
- Q: Model parametreleri**



$CBOW \rightarrow Q = N \times D + D \times \log_2(V)$   
 $Skip-Gram \rightarrow Q = C \times (D + D \times \log_2(V))$



## <sup>2</sup>Word2Vec

- **Softmax ile ilgili problemi çözmek için → “Hierarchical Softmax”**
- **Model performansını ölçmek için:**



**Daha sonra algebraic operation'lar ile (King-man+woman gibi) en similar vector bulunmuş. Eğer doğru vektör elde edilmiş ise doğru sayılmış.**

**Eğitimi optimize etmek için paralelde gradyan hesaplamaları yapılmış. Gradyanlar center bir sunucuda işlenmiş.**

<sup>1</sup>GloVe

\*Jeffrey Pennington, Richard Socher, Christopher D. Manning. 2014. GloVe: Global Vectors for Word Representation.

**Cooccurrence Matrix:** Kelimelerin birbirinin context window’unda görülme sayısını ölçer.

*“I like deep learning”*

*“I like NLP”*

|          | I | like | deep | learning | NLP |
|----------|---|------|------|----------|-----|
| I        | 0 | 2    | 1    | 0        | 1   |
| like     | 2 | 0    | 1    | 1        | 1   |
| deep     | 1 | 1    | 0    | 1        | 0   |
| learning | 0 | 1    | 1    | 0        | 0   |
| NLP      | 1 | 1    | 0    | 0        | 0   |

|          | I   | like | deep | learning | NLP |
|----------|-----|------|------|----------|-----|
| I        | 0   | 2    | 0.5  | 0        | 0.5 |
| like     | 2   | 0    | 1    | 0.5      | 1   |
| deep     | 0.5 | 1    | 0    | 1        | 0   |
| learning | 0   | 0.5  | 1    | 0        | 0   |
| NLP      | 0.5 | 1    | 0    | 0        | 0   |

$X_{ij}$  = i kelimesinin context window’unda j kelimesinin görülme sayısı

$C = 1/d$   
Uzaklığa bölünerek eklenir

- Bu yöntem ile çok sık kullanılan “the king” gibi yapılar semantic anlamı olmamasına rağmen yüksek skor üretir.

<sup>2</sup>GloVe

| Prob                    | solid                | gas                  | water                | fashion              |
|-------------------------|----------------------|----------------------|----------------------|----------------------|
| $P(k ice)$              | $1.9 \times 10^{-4}$ | $6.6 \times 10^{-5}$ | $3.0 \times 10^{-3}$ | $1.7 \times 10^{-5}$ |
| $P(k steam)$            | $2.2 \times 10^{-5}$ | $7.8 \times 10^{-4}$ | $2.2 \times 10^{-3}$ | $1.8 \times 10^{-5}$ |
| $P(k ice) / P(k steam)$ | 8.9                  | $8.5 \times 10^{-2}$ | 1.36                 | 0.96                 |

$$P_{ij} = P(j|i) = \frac{X_{ij}}{X_i}$$

- GloVe olasılıkları değil olasılık oranlarını kullanır.
- Birlikte görülen kelimelerin olasılığı büyük, görülmeyen kelimelerin olasılığı da küçüktür.
- Alakasız kelimeler için oran 1'e yakın çıkar: *fashion*
- İki kelime ile de alakalı kelimeler için oran 1'e yakın çıkar: *water*

$$F(w_i, w_j, \hat{w}_k) = \frac{P_{ik}}{P_{jk}} \xrightarrow[\text{Dönüşümler}]{\text{Baaaazı}} w_i^T \tilde{w}_j + b_i + b_j \approx \log(X_{ij})$$

**Loss Function for GloVe:**  $J = \sum_{i,j} f(X_{ij}) \underbrace{(w_i^T \tilde{w}_j + b_i + b_j - \log(X_{ij}))^2}_{\text{Bütün coocurrence'lara eşit ağırlık}}$

**f: weighting function**

$w_i, \tilde{w}_j, b_i, b_j \rightarrow$  Parametreler

# <sup>3</sup>GloVe

**Loss Function for GloVe:**  $J = \sum_{i,j} f(X_{ij}) (w_i^T \tilde{w}_j + b_i + b_j - \log(X_{ij}))^2$  **f: weighting function**

**f(0) = 0**

**f → azalmayan fonksiyon: Yüksek cooc → yüksek skor**

**f(x) büyük x için küçük olmalı → Çok sık geçen cooc algoritmayı domine etmesin.**

$$f(x) = \begin{cases} (x/x_{\max})^a & \text{if } x < x_{\max} \\ 1 & \text{otherwise} \end{cases}$$

**1. Adım → Tokenize**

**2. Co-occurrence matrix oluştur**

**3. non-zero  $X_{ij}$  çıkar**

**4. SGD/AdaGrad ile modeli eğit**

**5. Final Embedding olarak sadece  $w_i$  veya  $w_i - w_j$**

# FastText

Piotr Bojanowski, Edouard Grave, Armand Joulin, Tomas Mikolov. 2017. Enriching Word Vectors with Subword Information

- Tespit edilen temel problem daha önceki çalışmalarda her bir kelimenin bir vektör ile ifade edilmesi. Bu da Türkçe gibi kelime içerisinde ek bulunduran dillerde kelimelerin nadir geçmesine, iyi öğrenememesine sebep oluyor

|           |   |                                                                                |
|-----------|---|--------------------------------------------------------------------------------|
| geldi     | } | Aynı kök ancak her biri farklı vektör<br>bazı kelimeler daha nadir görülebilir |
| geliyor   |   |                                                                                |
| gelmekte  |   |                                                                                |
| geleyazdı |   |                                                                                |

- Bu sorunu çözmek için kelimeleri n-gram'ların toplamı olarak ifade etmişler.

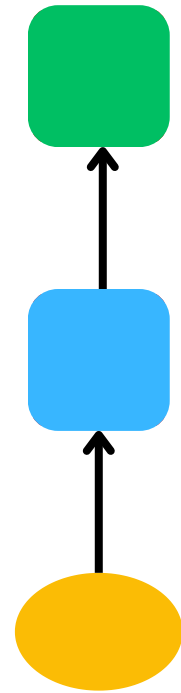
*kitap* → *<kitap>*

|     |        |                     |   |   |   |                        |
|-----|--------|---------------------|---|---|---|------------------------|
| <ki | .....→ | 1.Subword Embedding | } | + | → | <i>kitap</i> embedding |
| kit | .....→ | 2.Subword Embedding |   |   |   |                        |
| ita | .....→ | 3.Subword Embedding |   |   |   |                        |
| tap | .....→ | 4.Subword Embedding |   |   |   |                        |
| ap> | .....→ | 5.Subword Embedding |   |   |   |                        |

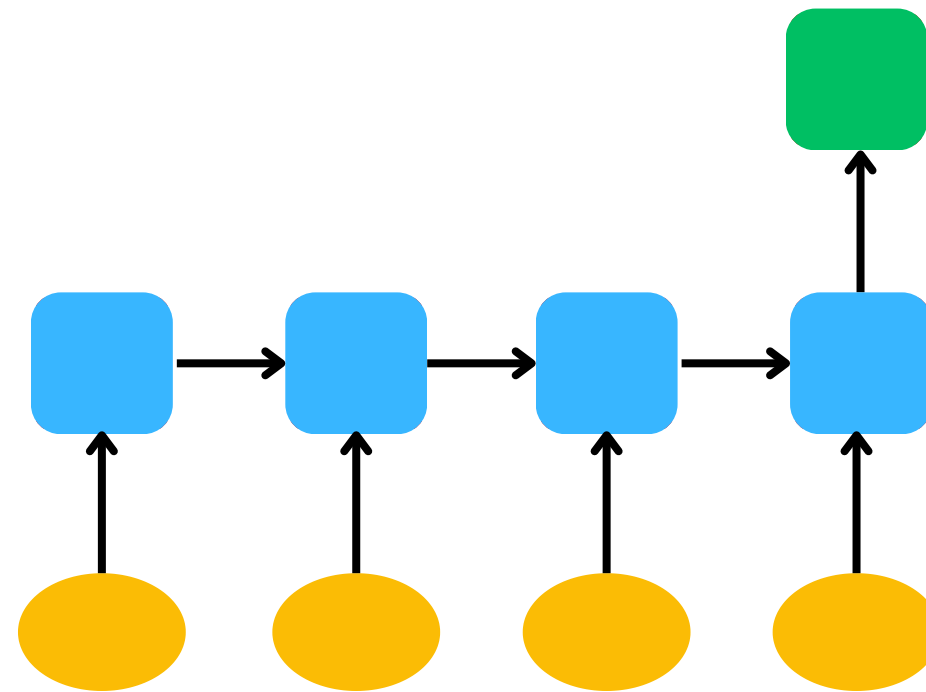
- Negative Sampling
- Hierarchical softmax
- Noise/Typo konusunda başarılı
- Out of Vocabulary (OOV) sorunu yok



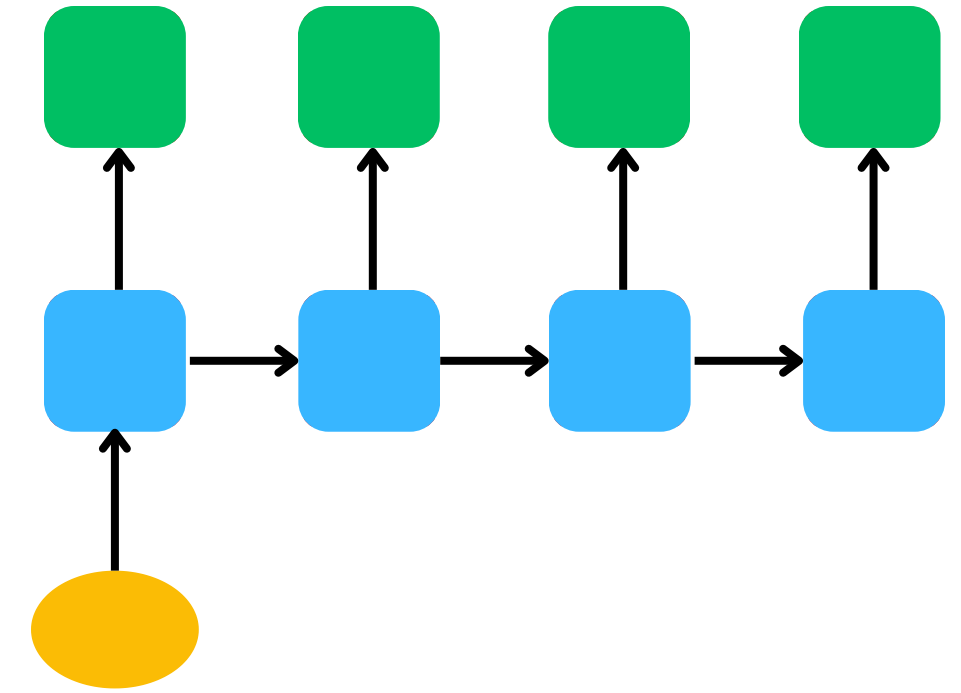
# <sup>1</sup>Sequence Modeling



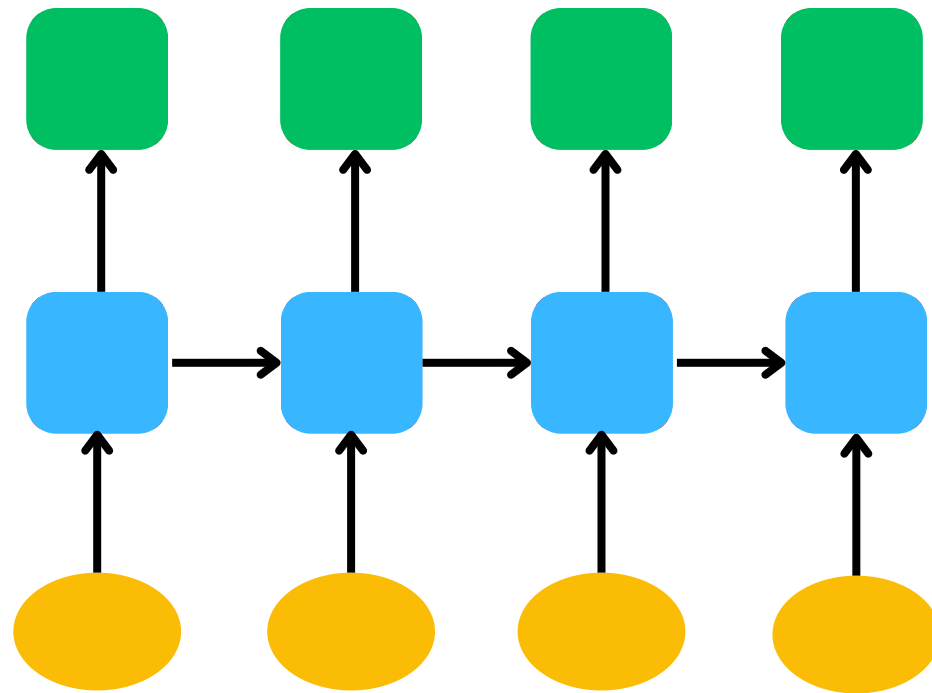
**one-to-one**



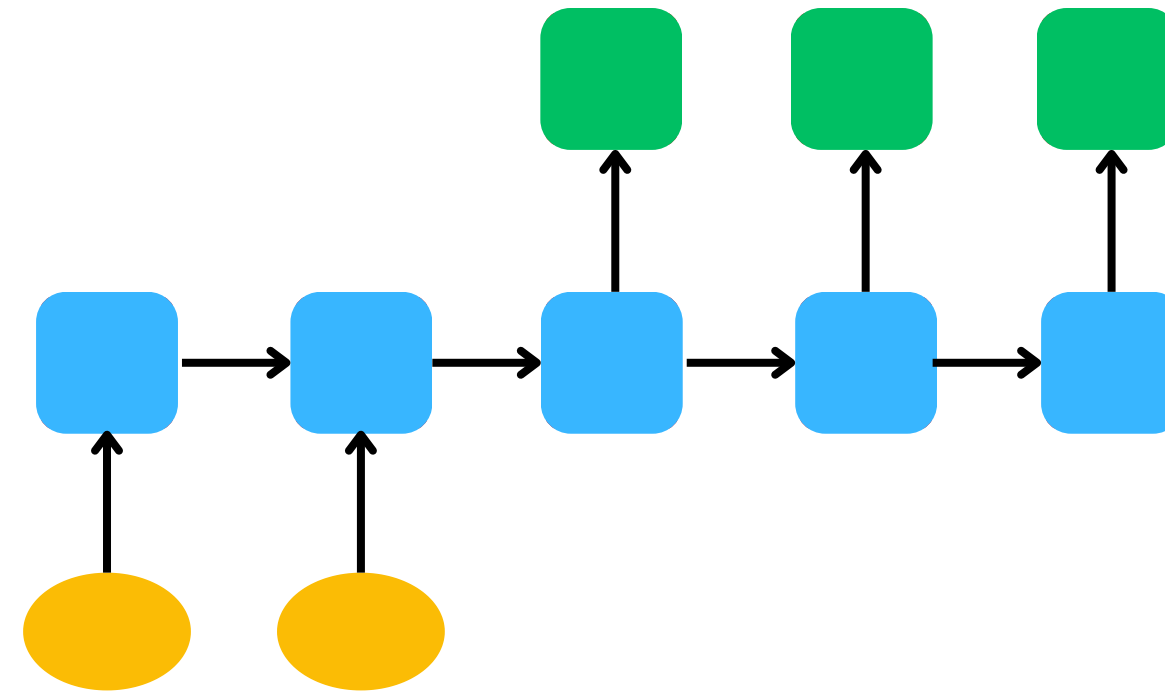
**many-to-one**



**one-to-many**

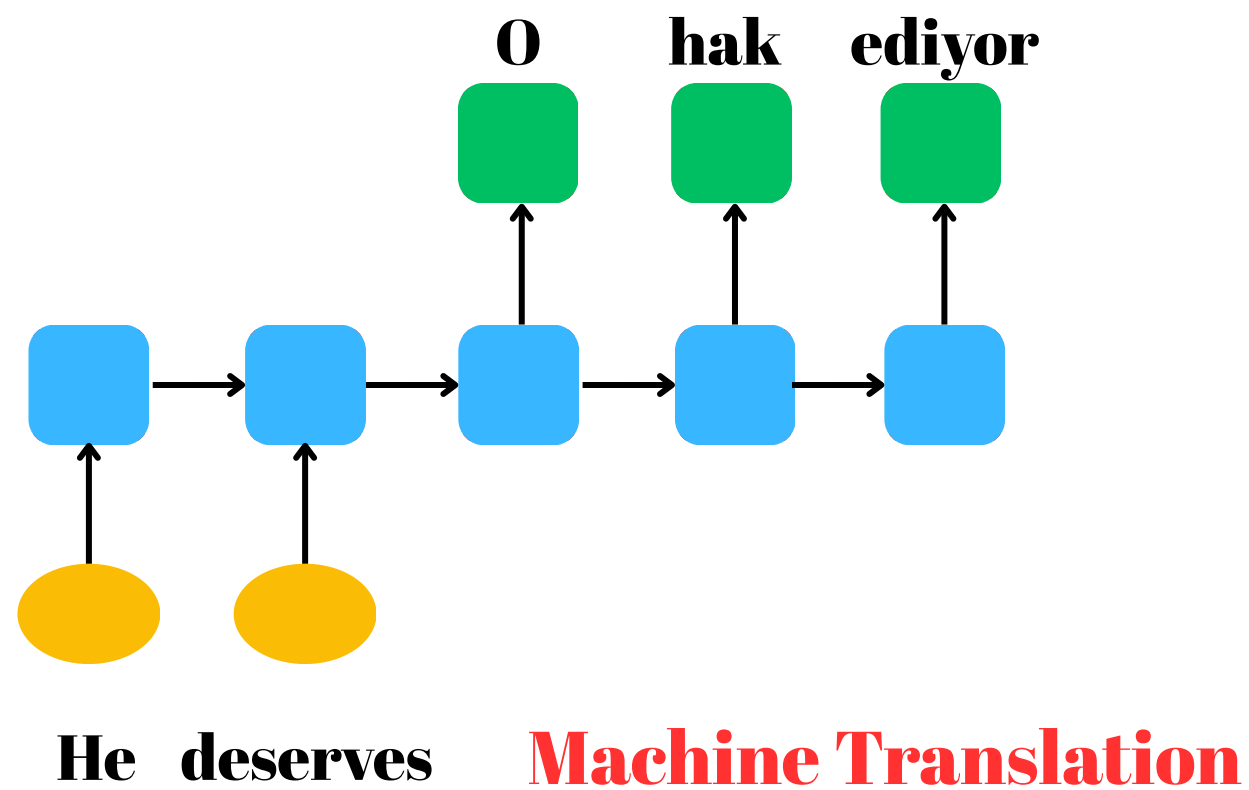
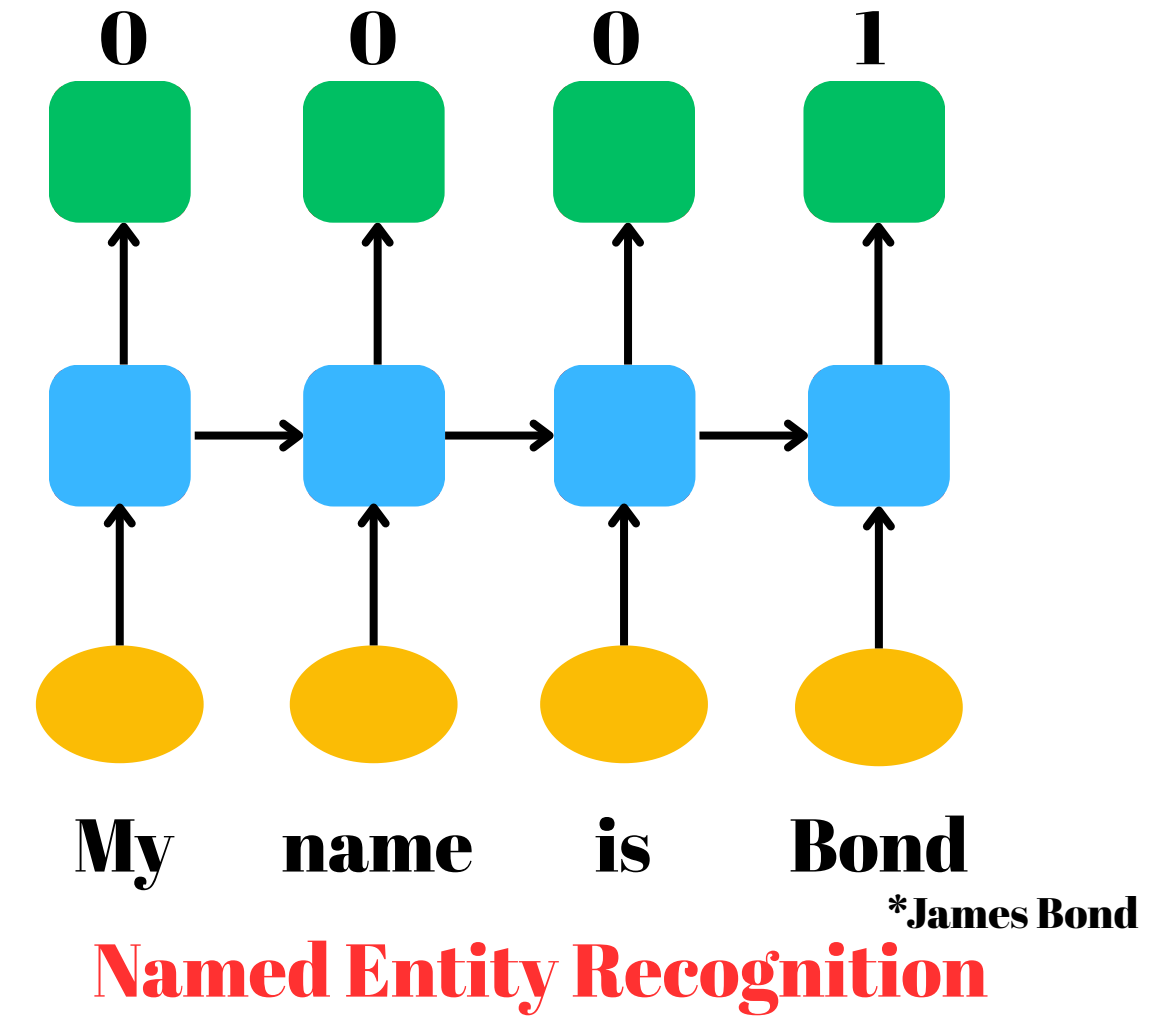
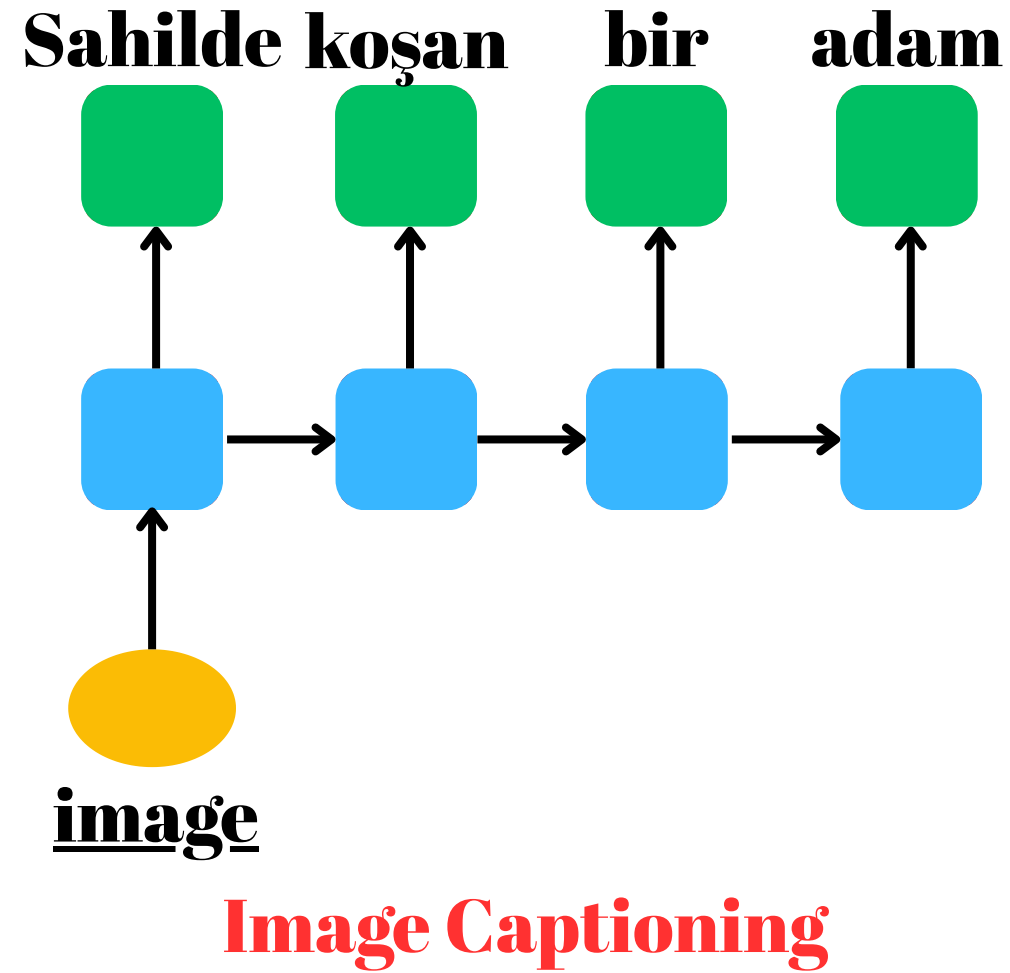
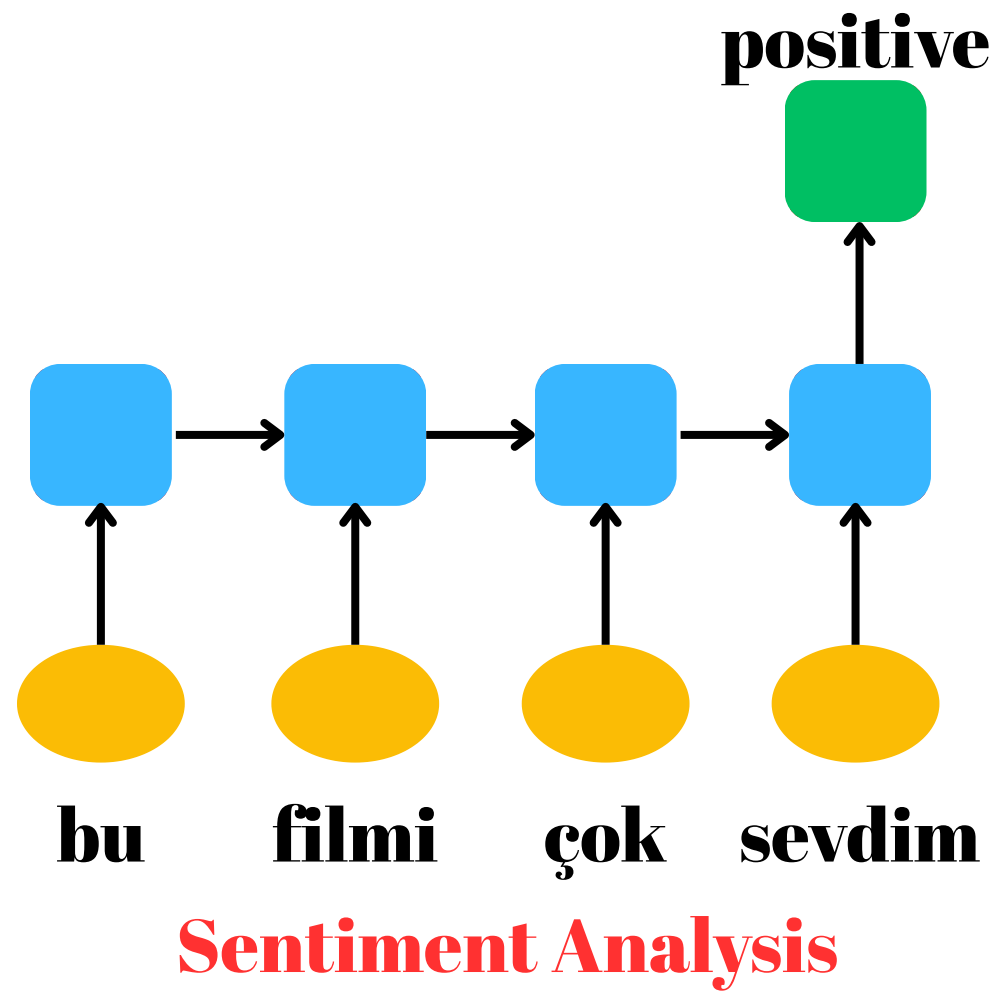


**many-to-many (aligned)**

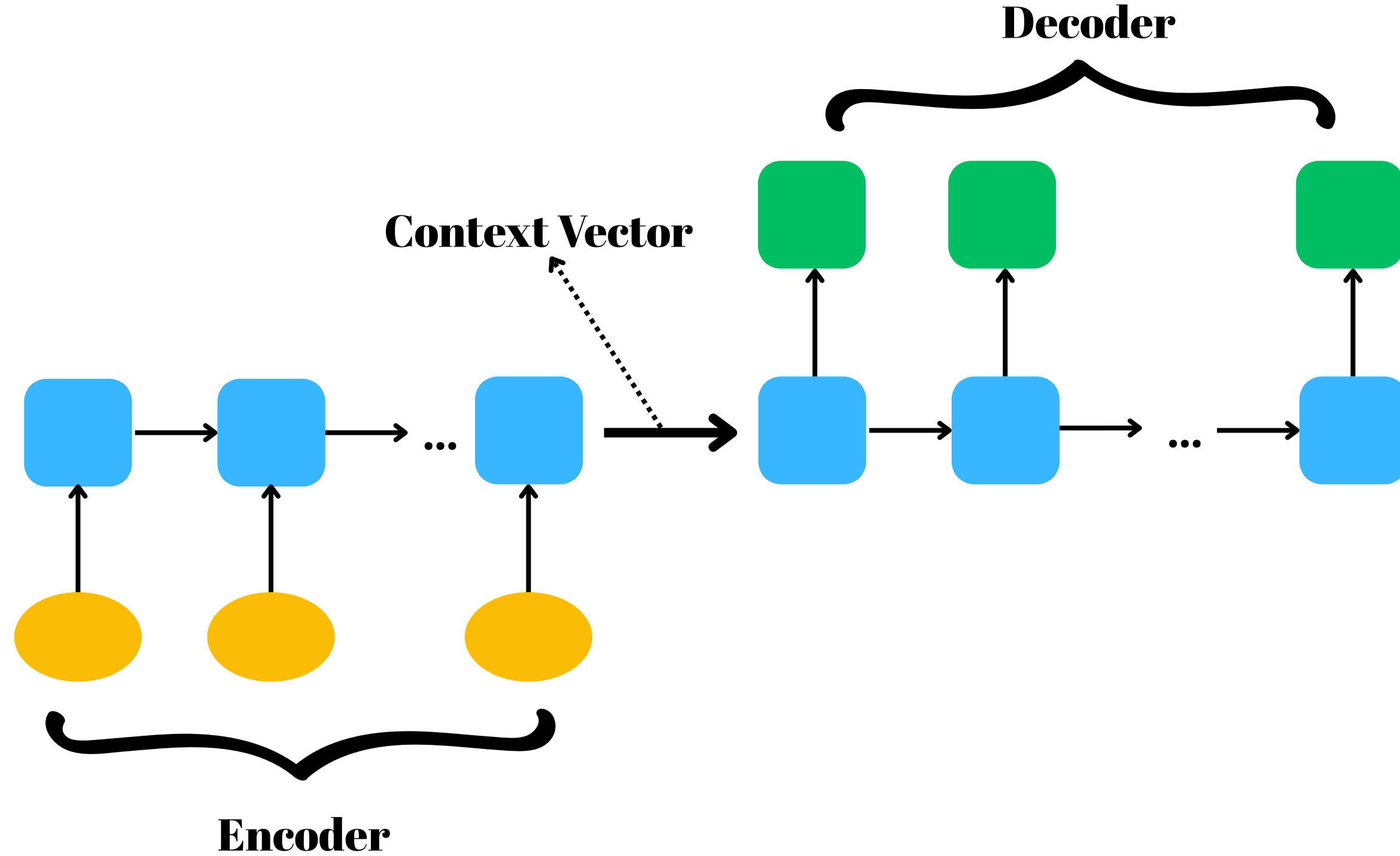


**many-to-many (unaligned)**  
**\*Seq2Seq**

## <sup>2</sup>Sequence Modeling



# Encoder-Decoder



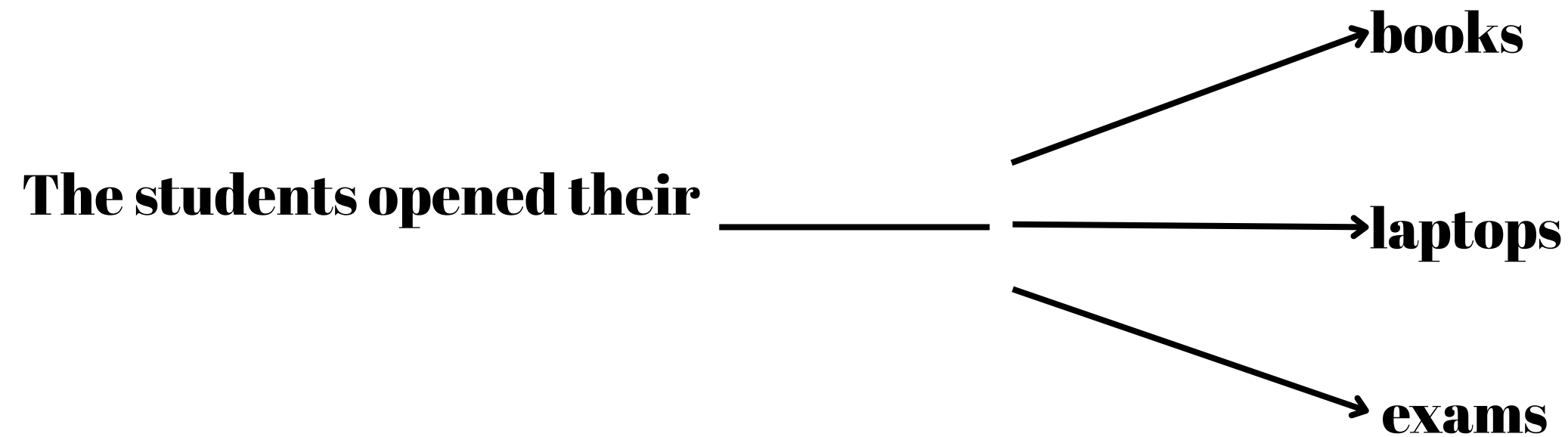
**Encoder:** Girdilerden anlam çıkar.

**Decoder:** Çıkarılan anlamı problemin çözümü için kullanır.



# Language Modeling

- Temel problem bir sequence'tan sonra gelen kelimeyi (ya da token) tahmin etmektir.
- Text generation, metin tamamlama, konuşma tanıma, LLM vb. bu yöntemeye dayanır.



*Instructor handed out the papers and the students opened their exams.*

*Instructor shared the link and the students opened their laptops.*

*Instructor stated the page number and the students opened their book.*

$P(\mathbf{x}^{t+1} \mid \mathbf{x}^t, \mathbf{x}^{t-1}, \dots, \mathbf{x}^1)$  —————> İlk t gözlemi gördükten sonra sıradaki (t+1). gözlemin olasılığı nedir

- Language Modeling yapılabilecek model türleri: N-gram , RNN , LSTM , GRU , Attention ...

# N-Gram

- Bitişik n tane token'ı temsil eder.

N = 1 → Unigram

N = 2 → Bigram

N = 3 → Trigram

*“Bu bir kısa cümledir”*

## Unigram

[  
“*Bu*”,  
“*bir*”,  
“*kısa*”,  
“*cümledir*”  
]

## Bigram

[  
“*Bu bir*”,  
“*bir kısa*”,  
“*kısa cümledir*”  
]

## Trigram

[  
“*Bu bir kısa*”,  
“*bir kısa cümledir*”  
]

- Char veya ses datası için de uygulanabilir.
- Küçük N: kısa context
- Büyük N: Sparse data

# <sup>1</sup>N-Gram Models

**$P(x^{t+1} | x^t, x^{t-1}, \dots, x^{t-n+1}) \longrightarrow$  N-Gram modelleri bunu tahmin eder. Son n-1 kelimeyi kullanır.**

**Formüldeki sadeleştirme Markov approximation**

# 1- Preprocess

## 2-Tokenization

## 3-N-Gram

***bu bir dil modeli***

**$\langle s \rangle \rightarrow$  başlangıç sinyali**

***<s> bu bir dil örneğidir </s>***

**</s> → bitiş sinyali**

**【 <s> bu bir bu bir dil bir dil modeli dil modeli </s> 】**

**[ <s> bu bir bu bir dil bir dil örneği dil örneği </s> ]**

## 2N-Gram Models

### 4-Count Tables

| Trigram                                                 |       | Bigram                                 |       | Unigram |       |
|---------------------------------------------------------|-------|----------------------------------------|-------|---------|-------|
| (w <sub>t-2</sub> , w <sub>t-1</sub> , w <sub>t</sub> ) | Count | (w <sub>t-2</sub> , w <sub>t-1</sub> ) | Count | Token   | Count |
| (<s>, bu, bir)                                          | 2     | (<s>, bu)                              | 2     | <s>     | 2     |
| (bu, bir, dil)                                          | 2     | (bu, bir)                              | 2     | </s>    | 2     |
| (bir, dil, modeli)                                      | 1     | (bir, dil)                             | 2     | bu      | 2     |
| (bir, dil, örneği)                                      | 1     | (dil, modeli)                          | 1     | bir     | 2     |
| (dil, modeli, )                                         | 1     | (dil, örneği)                          | 1     | dil     | 2     |
| (dil, örneği, </s>)                                     | 1     |                                        |       | modeli  | 1     |
|                                                         |       |                                        |       | örneği  | 1     |

Count Table için küçük projelerde “Hash Table”, büyük projelerde “Trie” kullanılabilir.

### 5-Probability

\*Markov Estimation

$$P(w_t \mid w_{t-(N-1)}^{t-1}) = \frac{\text{count}(w_{t-(N-1)}^t)}{\text{count}(w_{t-(N-1)}^{t-1})}$$

count(<s>, bu, bir) = 2

count(<s>, bu) = 2

}

P = 2/2 = 1.0

P(bir | <s> , bu) = 1.0

P(modeli | <s> , bu) = 0

...

P(</s> | <s> , bu) = 0

}

En yüksek olasılık  
“*bir*”  
kelimesi

<s> *bu bir*

# Smoothing

$$P(w_t \mid w_{t-(N-1)}^{t-1}) = \frac{\text{count}(w_{t-(N-1)}^{t-1})}{\text{count}(w_{t-(N-1)}^{t-1})}$$

.....>0 olabilir

Corpus içerisinde daha önce görülmemiş diziler 0 olur.  
Bu tanımsızlık ya da bütün cümlelerin olasılığının 0 çıkması gibi durumlara yol açar.

Laplace (Add 1) Smooting:

$$P(w_t \mid w_{t-(N-1)}^{t-1}) = \frac{\text{count}(w_{t-(N-1)}^{t-1}) + 1}{\text{count}(w_{t-(N-1)}^{t-1}) + V}$$

+1 sıfır olmanın önüne geçer +V ise normalize eder.

V: Vocab size

1 yerine k

V yerine kV

} **Add K**

| Vocab | Bigram  | C |
|-------|---------|---|
| ben   | ben sen | 1 |
| sen   | ben o   | 0 |
| o     | ben ben | 0 |

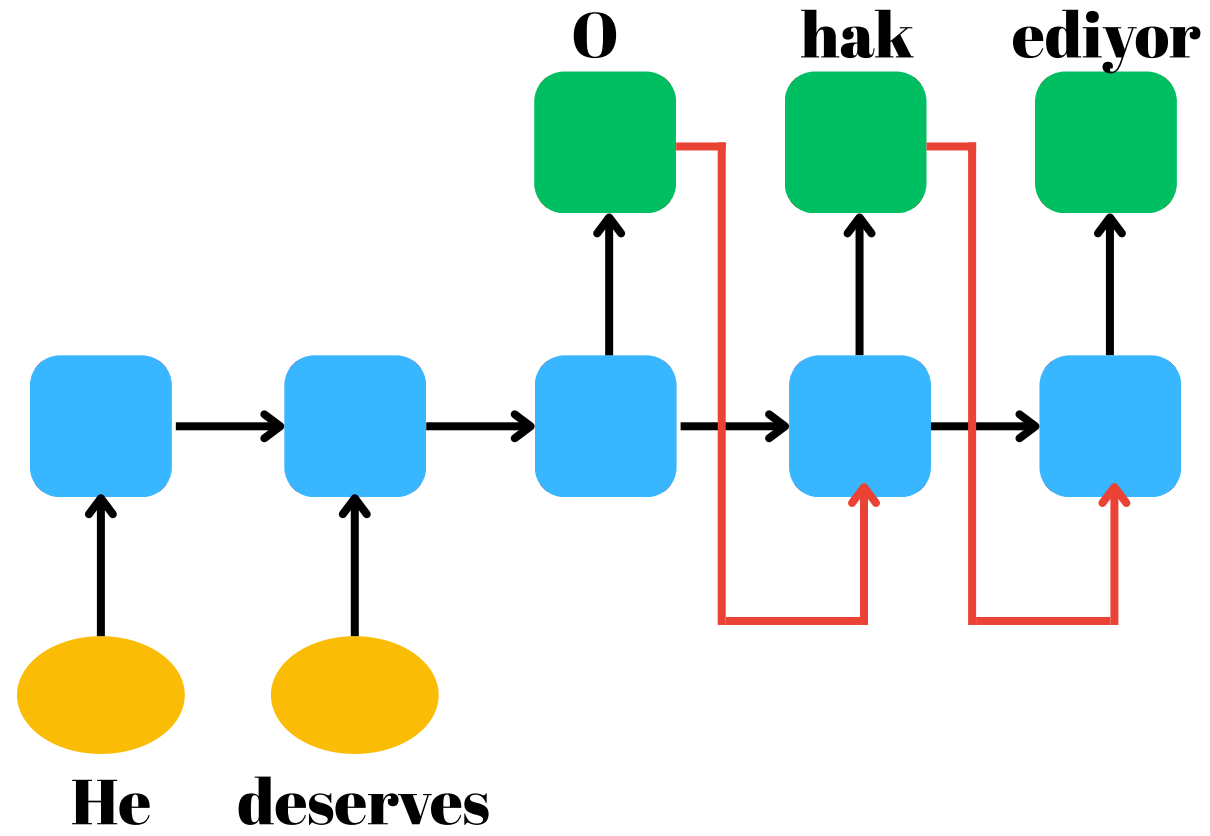
Yeni dizi → “ben”

→MLE: P(o | ben)=0

→Smoothing: P(o | ben)=(0+1)/(2+3)=1/5



# Autoregressive Generation

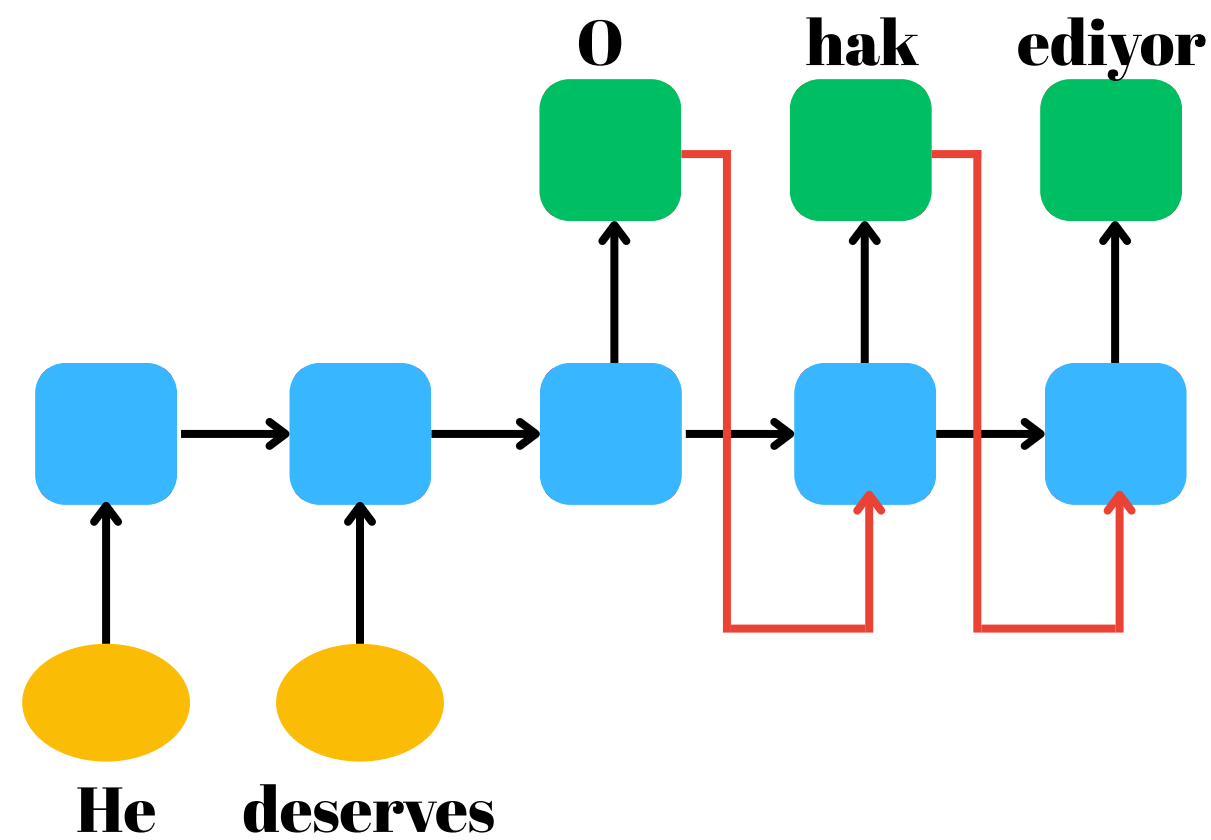


**Mavi kutuların her biri bir input istiyor**

**N-gram her adımda son n-1 kelimeyi istiyor**

**Algoritma ilk başta birinci output'u tahmin ediyor. Sonrasında bu outputu input gibi kullanarak 2. kelimeyi tahmin ediyor ...**

# Greedy Search



$$y_t = \arg \max P(y_t \mid y_1^{t-1})$$

Greedy Search ilk başta olasılığı en yüksek olan 1. kelimeyi ( $y_1$ ) seçer. Sonrasında  $y_1$  kullanarak olasılığı en yüksek olan  $y_2$  seçer...

Greedy search lokal olarak en iyi token'ı seçer ama global olarak en iyi sequence'ı garanti etmez.

Nathan haftasonu tatile gidiyor

.....>Nathan is going to be on vacation this weekend.

.....>Nathan is going on vacation this weekend.

| S1 Scores | S2 Scores |
|-----------|-----------|
| 0.4       | 0.4       |
| 0.45      | 0.4       |
| 0.25      | 0.5       |
| 0.3       | 0.6       |

# Beam Search

Nathan is going

\_\_\_\_\_

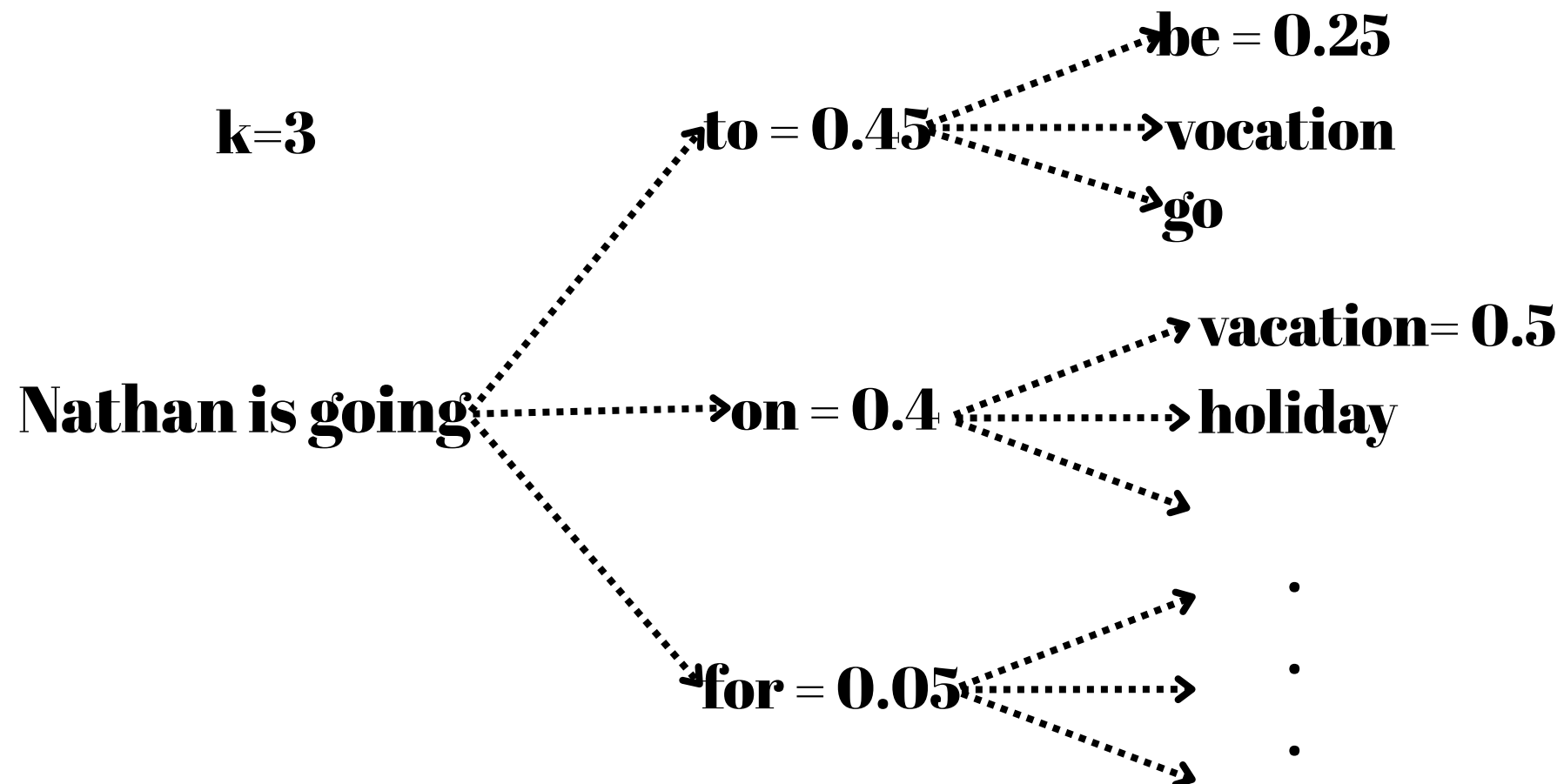
3 boşluğa gelebilecek bütün dizileri bulalım  
Hepsinin olasılığını hesaplayalım  
En yüksek olasılıklı diziyi alalım

İşlem yükü

$$\hat{y}_{1:T} = \arg \max_{y_{1:T}} P(y_{1:T}) = \arg \max_{y_{1:T}} \prod_{t=1}^T P(y_t | y_1^{t-1})$$

Beam Search ne tek tek bakar ne de bütün olasılıklara bakar. İkisinin arasında bir denge kurar.  
Her adımda k kadar dizi değerlendirilir.

k=3



en yüksek 3'ü seç

.....>Nathan is going on vacation

.....>Nathan is going to be

.....>Nathan is going on holiday



# Length Normalization

**$0.2 \times 0.17 \times 0.04 \times 0.2 \times \dots \rightarrow$  Numerical underflow**

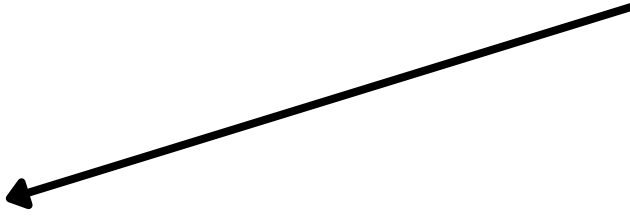
**Çok küçük sayıları sürekli çarpınca bir süre sonra 0'a çok yaklaştığı için underflow olur.**

**Underflow'un önüne geçmek için logaritma alırız.**

$$\log(a \cdot b) = \log a + \log b$$

$$\text{score}(y_{1:T}) = \frac{1}{T^\alpha} \sum_{t=1}^T \log P(y_t | y_1^{t-1})$$

**Normalization**



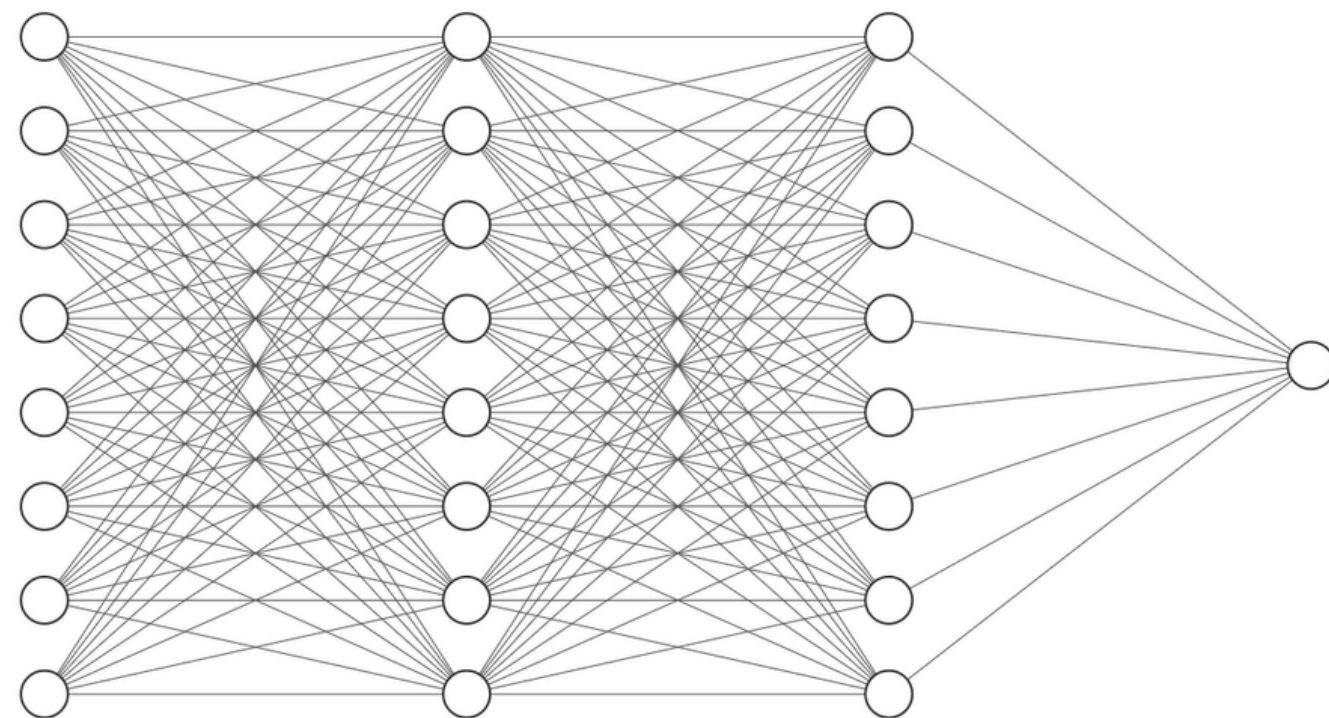
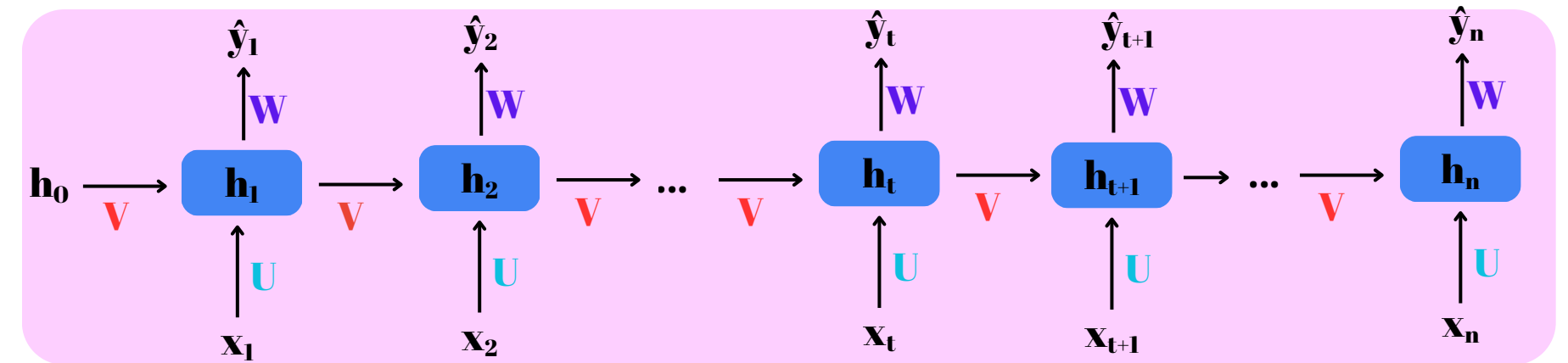
**alpha = 1.0  $\rightarrow$  Full normalization**

**alpha = 0.0  $\rightarrow$  No normalization**

**alpha = 0.7  $\rightarrow$  Balance**

# Sequence Models

| Neural Network                                                  | Sequence Models                                                             |
|-----------------------------------------------------------------|-----------------------------------------------------------------------------|
| Neural network yapıları her bir girdiyi bağımsız değerlendirir. | Sequence modeller önceki girdilerin bilgilerini de kullanarak tahmin yapar. |
| Bilgi paylaşımı yoktur                                          | Geçmişteki bilgiyi bellekte tutar.                                          |
| Parameter Sharing yoktur. Sabit bir girdi boyutu vardır.        | Parameter Sharing vardır. Farklı girdi boyutları ile işlem yapabilir.       |

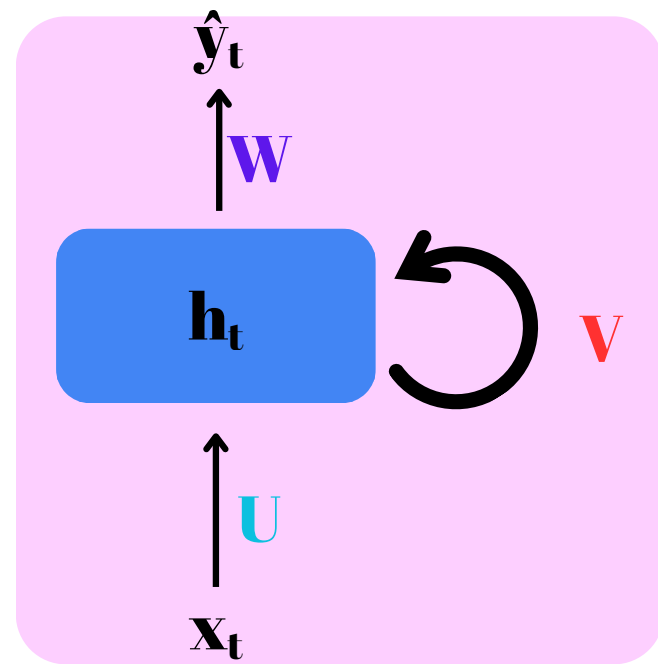
Input Layer  $\in \mathbb{R}^8$ Hidden Layer  $\in \mathbb{R}^8$ Hidden Layer  $\in \mathbb{R}^8$ Output Layer  $\in \mathbb{R}^1$ 

# <sup>1</sup>Recurrent Neural Networks

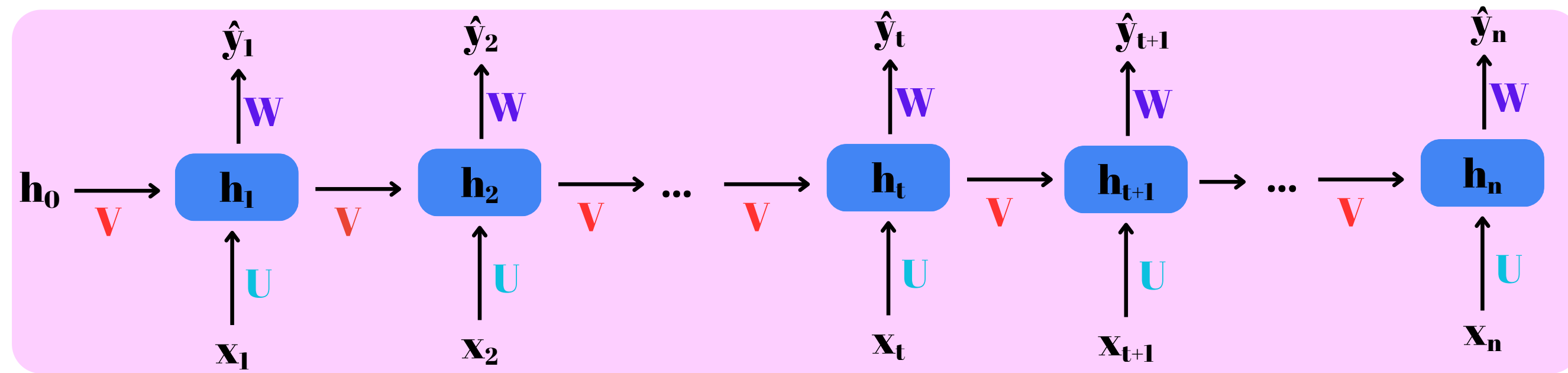
Sequence Data için özelleşen bir Sinir ağı türüdür. Recurrent bağlantıları vardır.

Sequence içindeki inputları sıra ile işler. Input'lar arasında **parameter sharing** vardır. Parameter sharing modeling farklı uzunluktaki girdileri işlemesine olanak sağlar.

Input sırası göz önünde bulundurulur. Önceki inputlar sonrakileri etkiler.



**Folded**



*“Bu”*

*“bir”*

*“yapay”*

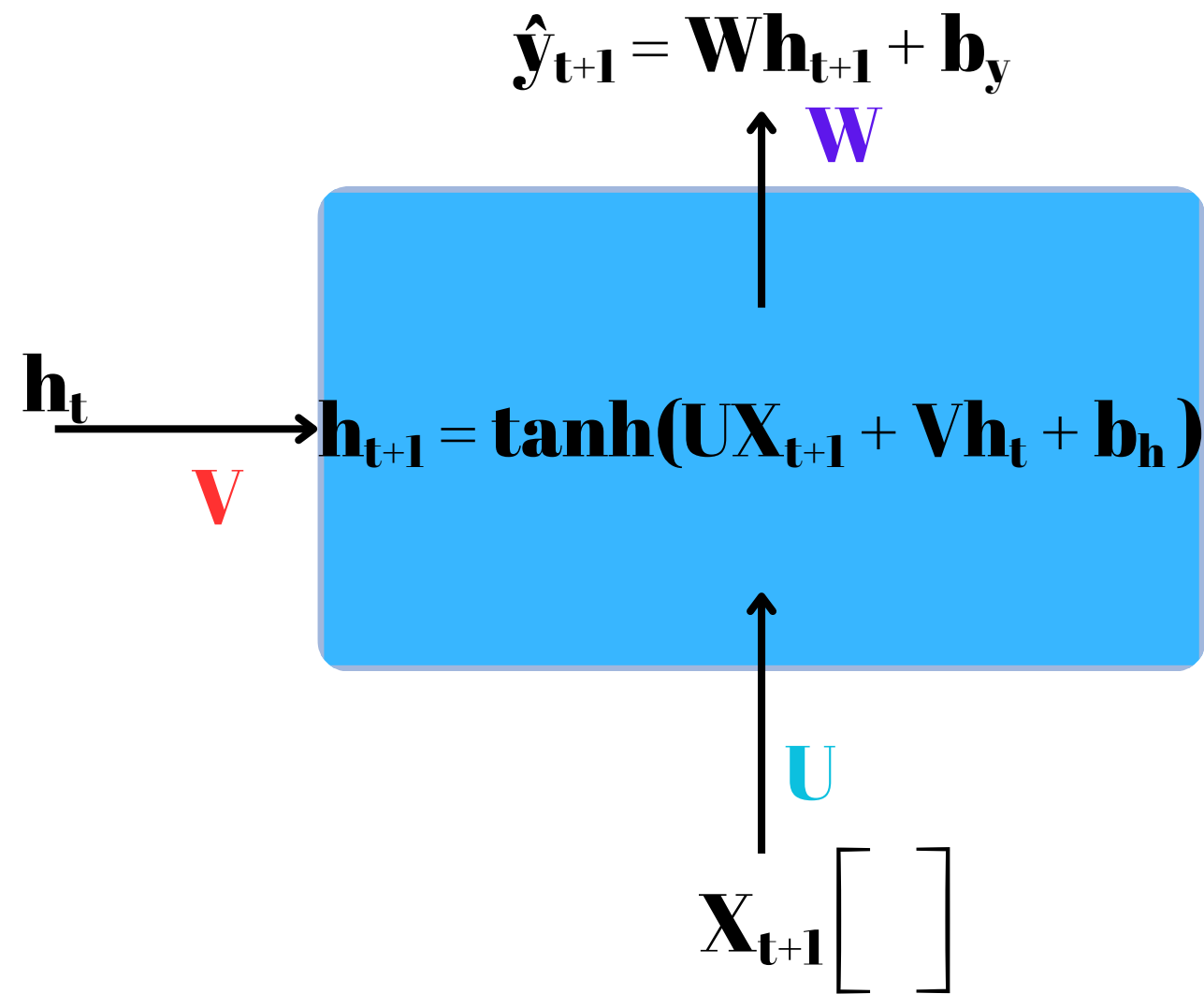
*“zeka”*

*“.”*

**Unfolded**

Burada aktarılan bilgi olan  $h_t$  o ana kadar olan girdiler için bir [kayıplı] özet niteliğindedir. (Goodfellow & Bengio)

## <sup>2</sup>Recurrent Neural Networks



**X herhangi bir andaki/sıradaki input**

**U input'tan anlam çıkarmak için kullanılan parametre**

**V bir öndeki state'ten gelen bilgiden anlam çıkaran parametre**

**W output oluşturmak için kullanılan parametre**

**X: (*input*,1)**

**h: (*hidden*,1)**

**U: (*hidden,input*)**

**V: (*hidden,hidden*)**

**W: (*output,hidden*)**

**input: 300**

**output: 1**

**hidden: 500**

**X: (300,1)**

**h: (500,1)**

**U: (500,300)**

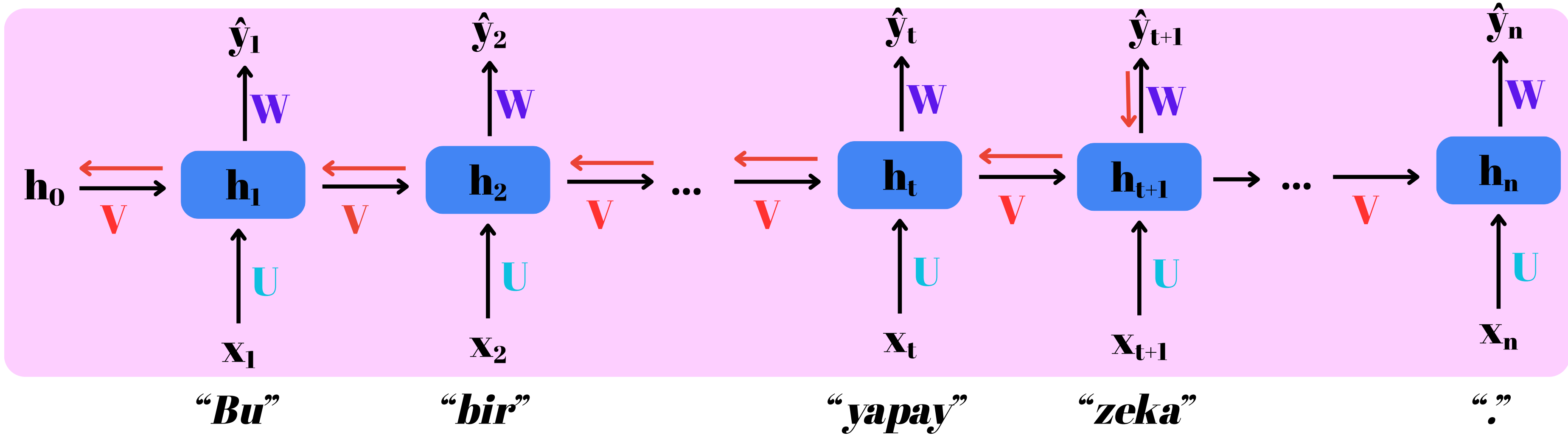
**V: (500,500)**

**\* W: (1,500)**

- $\mathbf{h}_0$  zero vector ya da random initialized olabilir.
- $\mathbf{y}$  hesaplanırken probleme bağlı olarak sigmoid-softmax gibi activation function'lar uygulanabilir.

### <sup>3</sup>Recurrent Neural Networks

**Backpropagation Through Time (BPTT):** Hidden stateler arasında backpropagation yapılmasını ifade eder.



$$\text{Total Loss} = l_1 + l_2 + \dots l_n$$

- $y_t$  ve Loss problem özelinde değerlendirilen bir durumdur.
- Örnek olarak her bir kelimeyi özel isim mi değil mi şeklinde etiketleyen bir sistem geliştirelim. Bu durumda  $y_t$  0-1 binary classification problemi olur. Loss function da BCE olarak seçilebilir.
- Loss hesaplandıktan sonra kısmi türev ve chain rule ile geriye doğru yayılabilir.
- Pratikte sadece son 10-20 hidden state için backprop yapılabilir.



## <sup>4</sup>Recurrent Neural Networks

**RNN ile ilgili problemler:**

- 1. Pratikte sonraki bilgiler outputu domine edebiliyor.**
- 2. Bir andaki hidden state bir önceki output'a bağlı olduğu için paralel işlem mümkün değil bu yüzden eğitim uzun sürebiliyor.**
- 3. Çok uzun sequence üstünde eğitim yapılırsa vanishing/exploding gradient problemi ortaya çıkabiliyor.**

*The dog, which barked excessively at night and frightened the old woman, is now silent.*

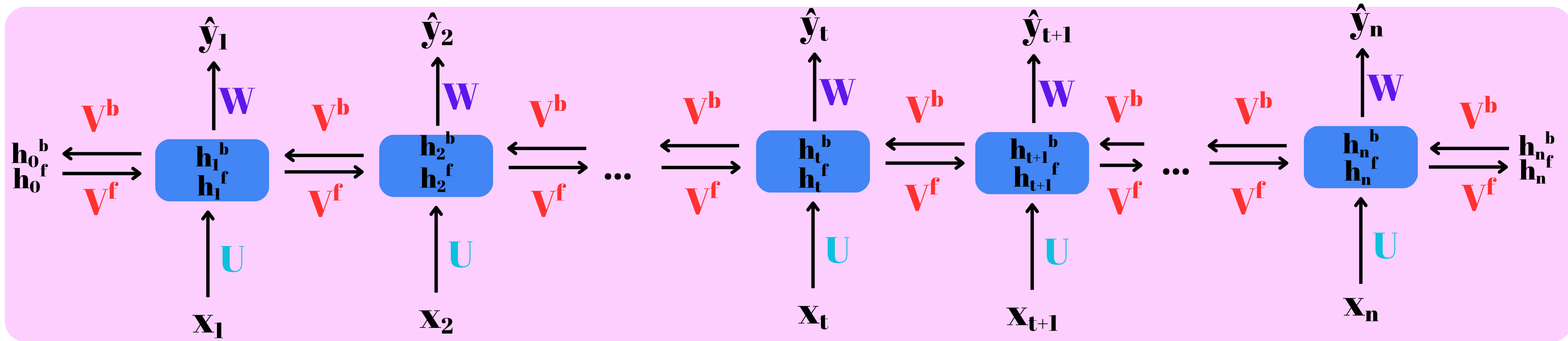
*The dogs, which barked excessively at night and frightened the old woman, are now silent.*

# Bidirectional RNN

RNN'ler sadece geçmişten bilgi getirebilir. Bu durum bazı problemlerde performansı düşürebilir.

“Özgür makine öğrenmesi kursu hazırlıyor”  
 “Özgür kalmak için gereken tek şey, umut ve bir miktar sabırdır”

} NER probleminde  
 RNN sıkıntı yaşar

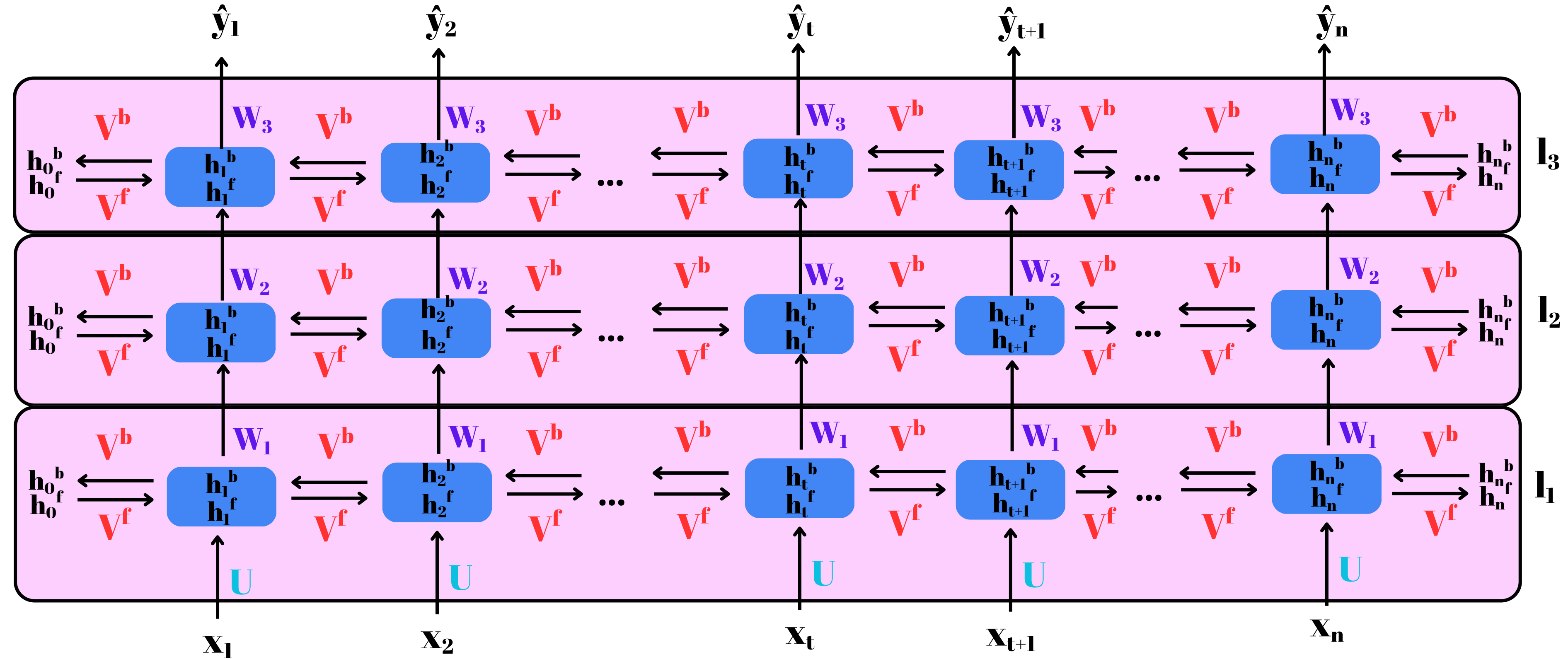


NER, Disambiguity, Protein sequence, Speech data vb.'de başarılıdır.

BRNN'ler language modeling için kullanılamaz. Çünkü bütün sequence'a ihtiyaç vardır.

# Deep RNN

Deep RNN farklı katmanlarda farklı özellikler öğrenebilir.

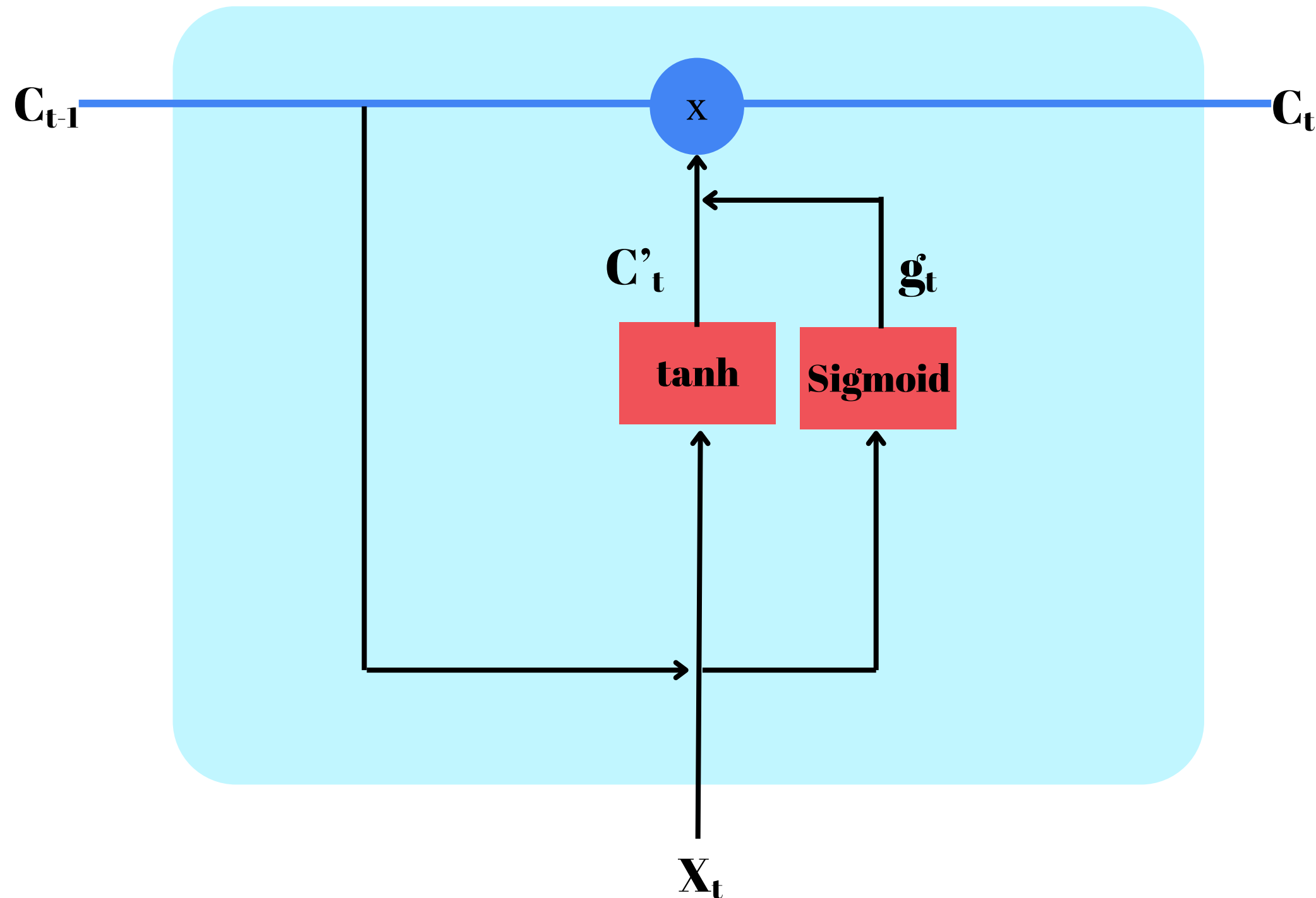


Her katman ( $l$ ) için  $V, W, h$  farklıdır



# <sup>1</sup>Gated Recurrent Unit (GRU)

**GRU kapılama (gate) mantığı ile çalışır. Bilgilerin hafızada ne kadar tutulacağını öğrenebilir.**



$$g_t = \sigma(W_g x_t + b_g)$$

$$\tilde{C}_t = \tanh(W_c x_t + b_c)$$

$$C_t = g_t \odot \tilde{C}_t + (1 - g_t) \odot C_{t-1}$$

## <sup>2</sup>Gated Recurrent Unit (GRU)

Burada gate (g) her bir context deęişimini algılayan bir anahtar gibidir.

g 300 boyutlu bir vektör ise bu algoritmanın farklı context'ler için 300 farklı anahtar öğrenebileceęi anlamına gelir. Örneęin bir tanesi tekil/çoęul , bir tanesi zaman, bir tanesi cinsiyet vb.

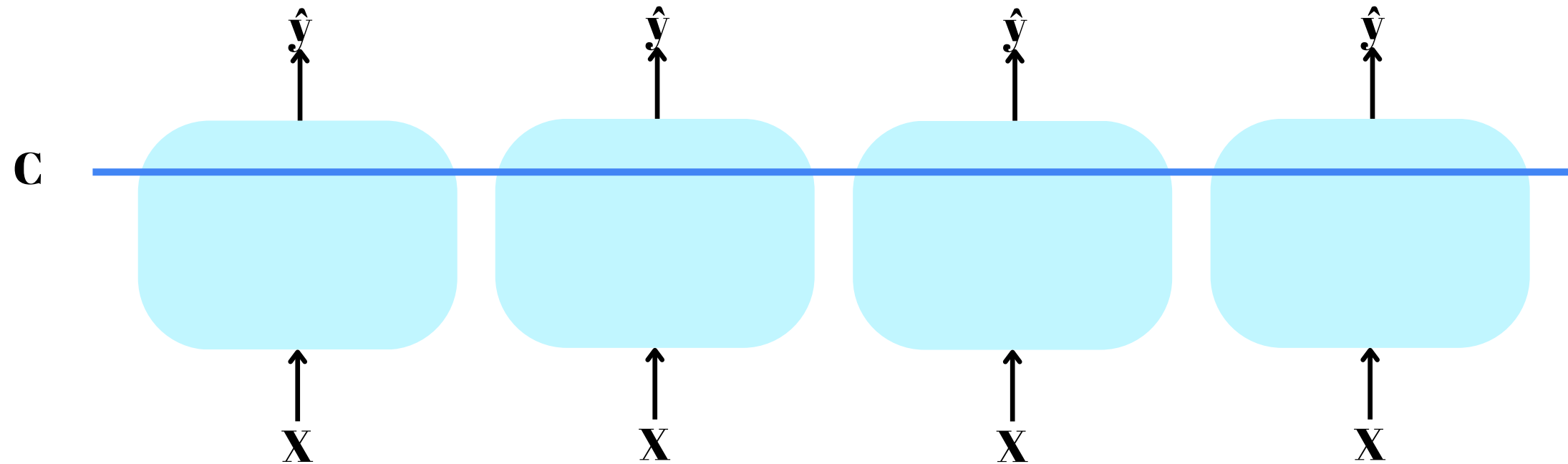
*The dog, which barked excessively at night and frightened the old woman, is now silent.*

$g = 0$

$g = 1$

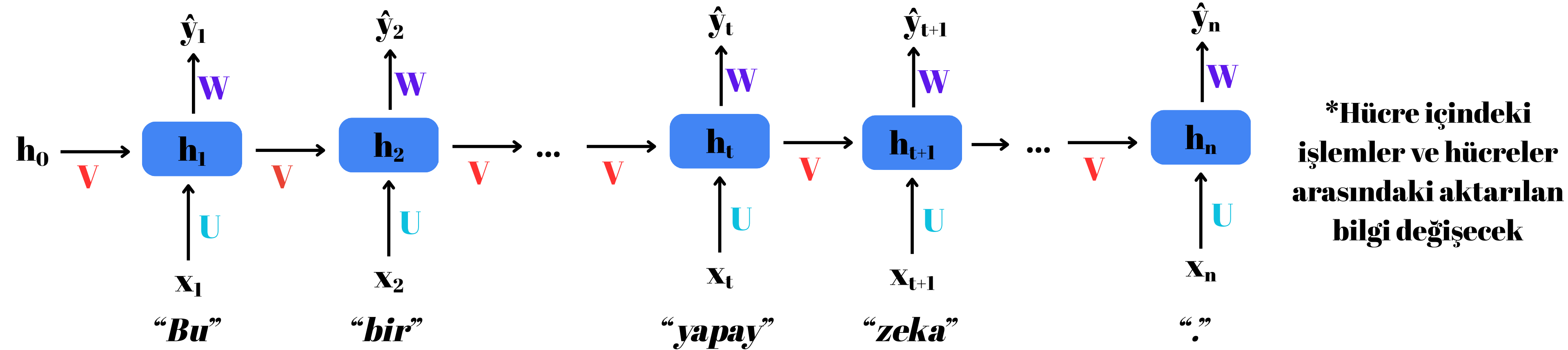
$$C_t = g_t \odot \tilde{C}_t + (1 - g_t) \odot C_{t-1}$$

- GRU kronolojik olarak LSTM'den sonra çıkmıştır ancak sadeleştirilmiş bir versiyon olduęu için LSTM daha karmaşıktır.
- Slaytta gösterilen GRU sadeleştirilmiştir. Literatürde birden fazla kapılı farklı GRU hücreleri bulunur.



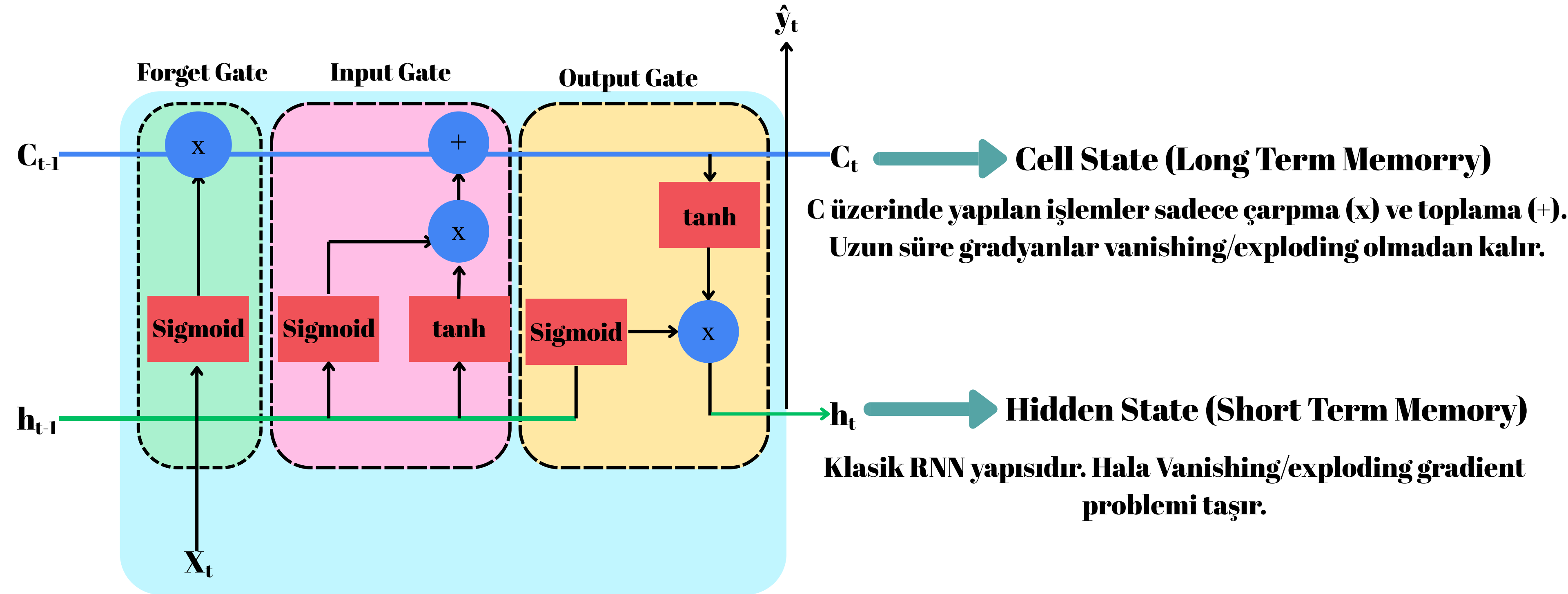
# <sup>1</sup>Long-Short Term Memory (LSTM)

Mantık olarak RNN ile benzerdir ancak her bir hücre içinde gerçekleşen işlemler farklıdır



- **Vanishing gradient problemini çözmeyi amaçlar. Bunun için hem kısa süreli (Short Term) hem uzun süreli (Long Term) bellek tutar.**

## <sup>2</sup>Long-Short Term Memory (LSTM)

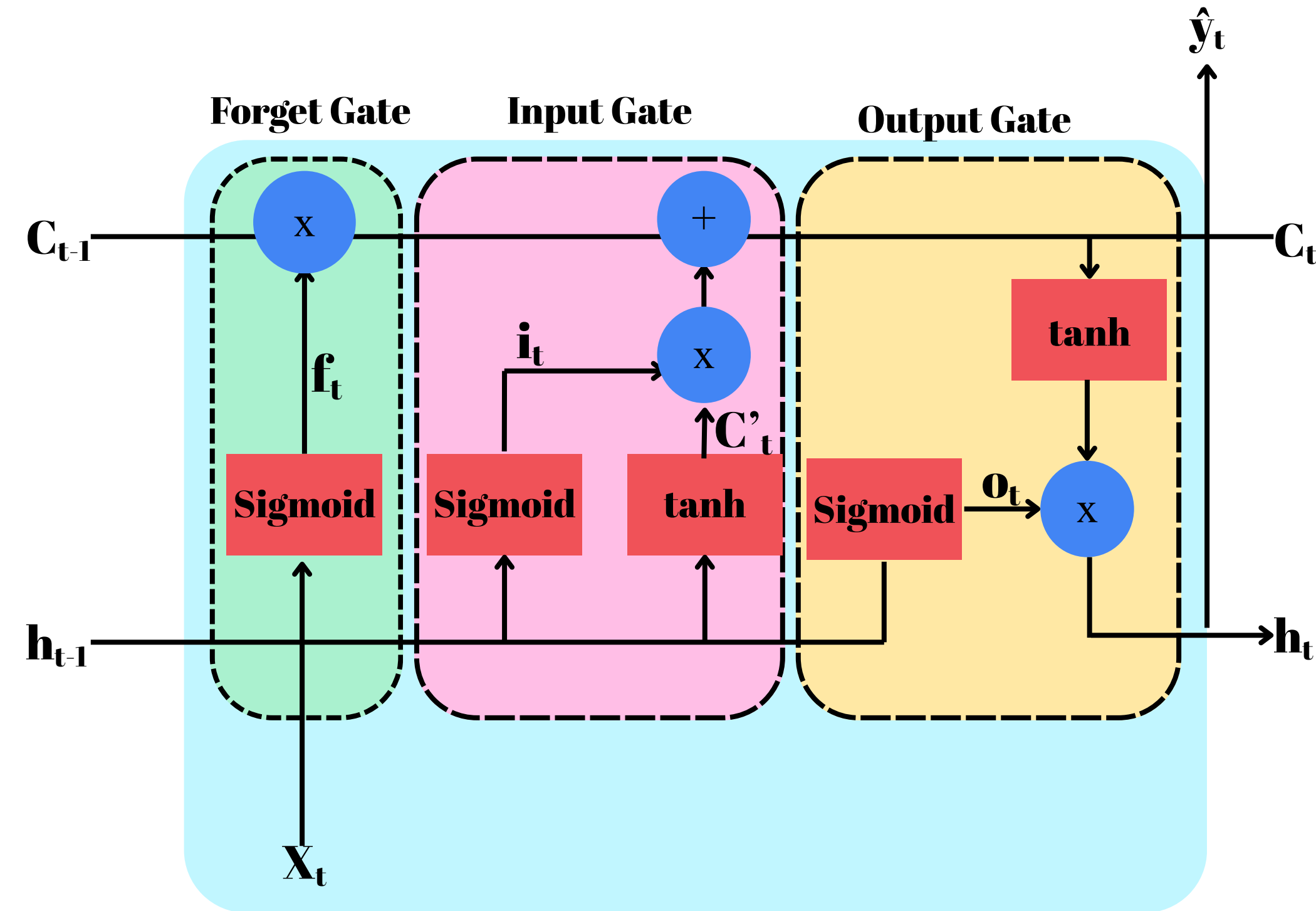


**Forget Gate** C'nin ne kadar hatırlanacağını belirler.

**Input Gate** yeni gelen bilginin C'ye ne kadar yazılacağına karar verir.

**Output Gate** C'nin bilgisinin ne kadarının dışarıya verileceğini belirler.

# <sup>3</sup>Long-Short Term Memory (LSTM)



$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)$$

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i)$$

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)$$

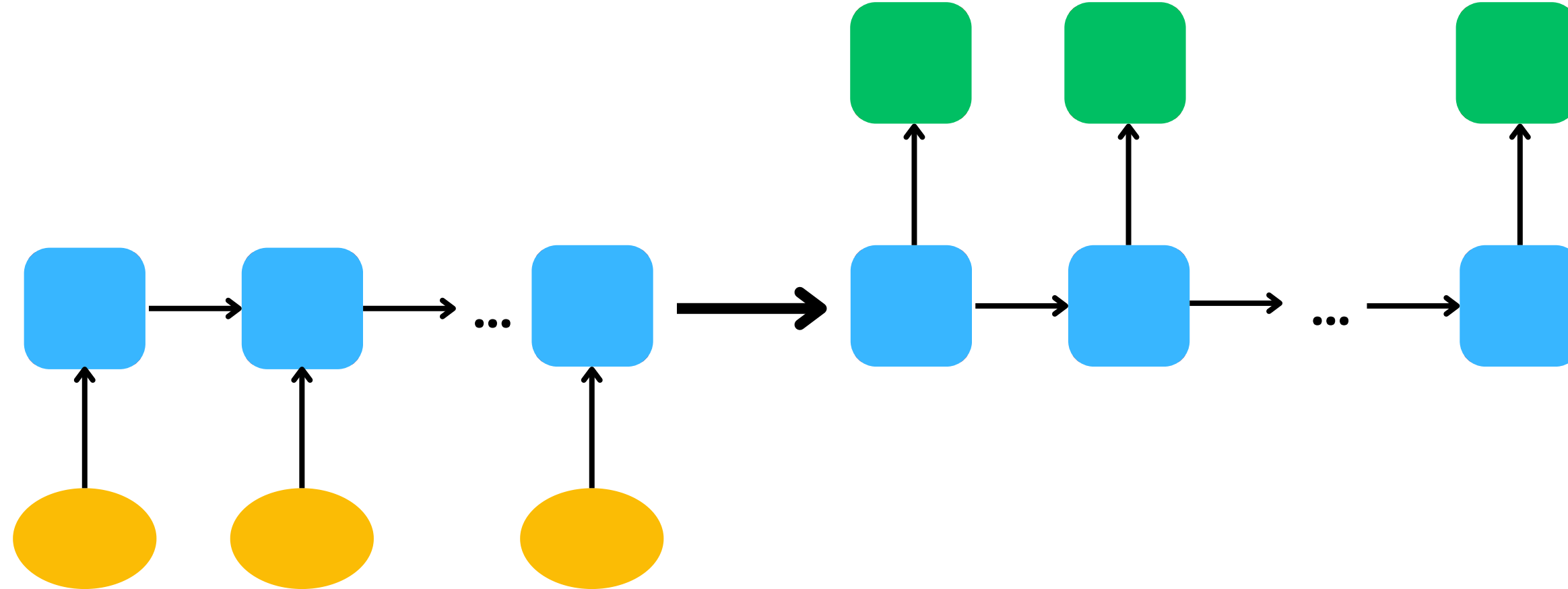
$$h_t = o_t \odot \tanh(c_t)$$

$$\hat{y}_t = \sigma(W_y h_t + b_y)$$



# Attention Model Intuition

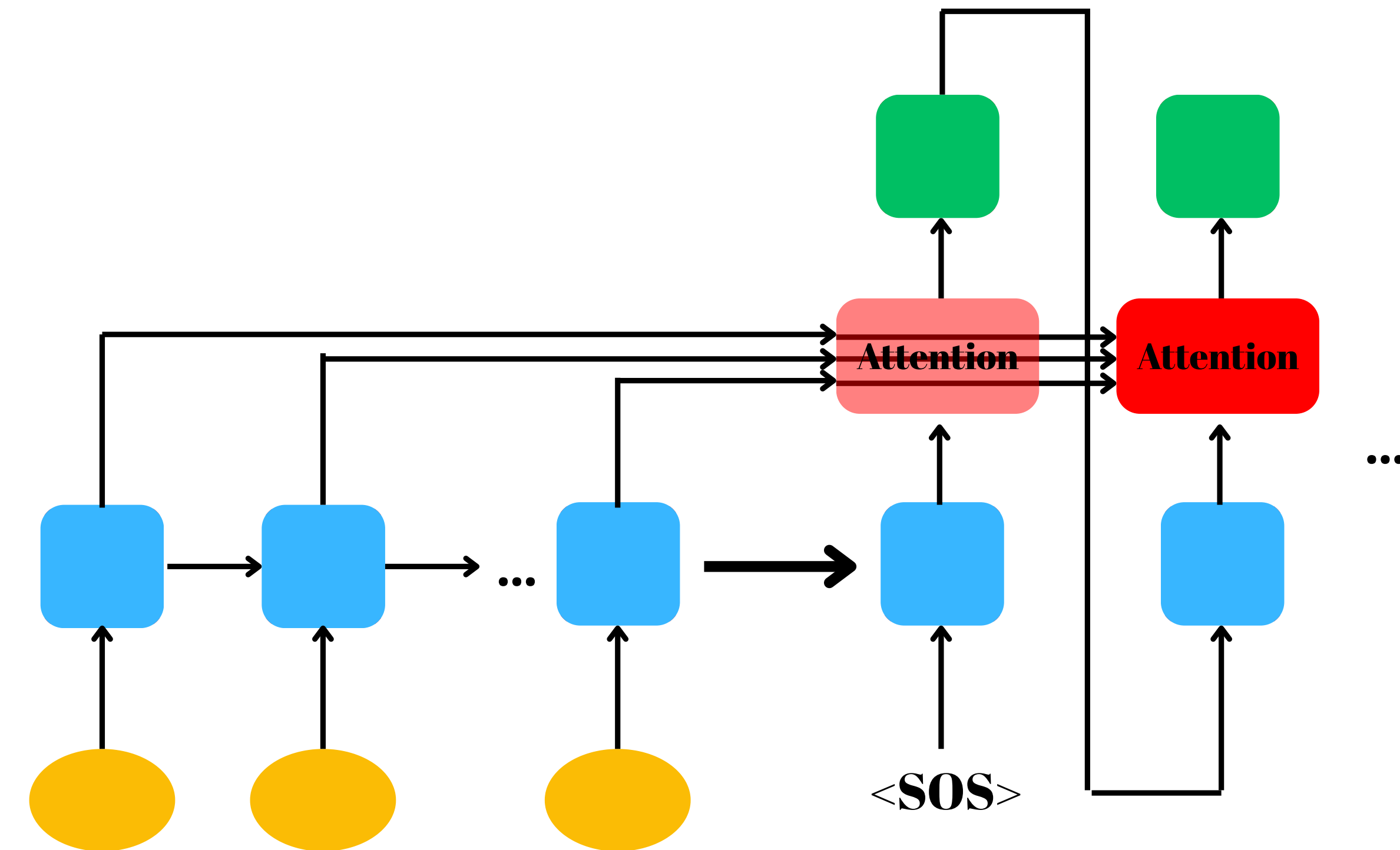
**Geceleyin aşırı derecede havlayarak yaşlı kadını korkutan köpek artık sessiz.**



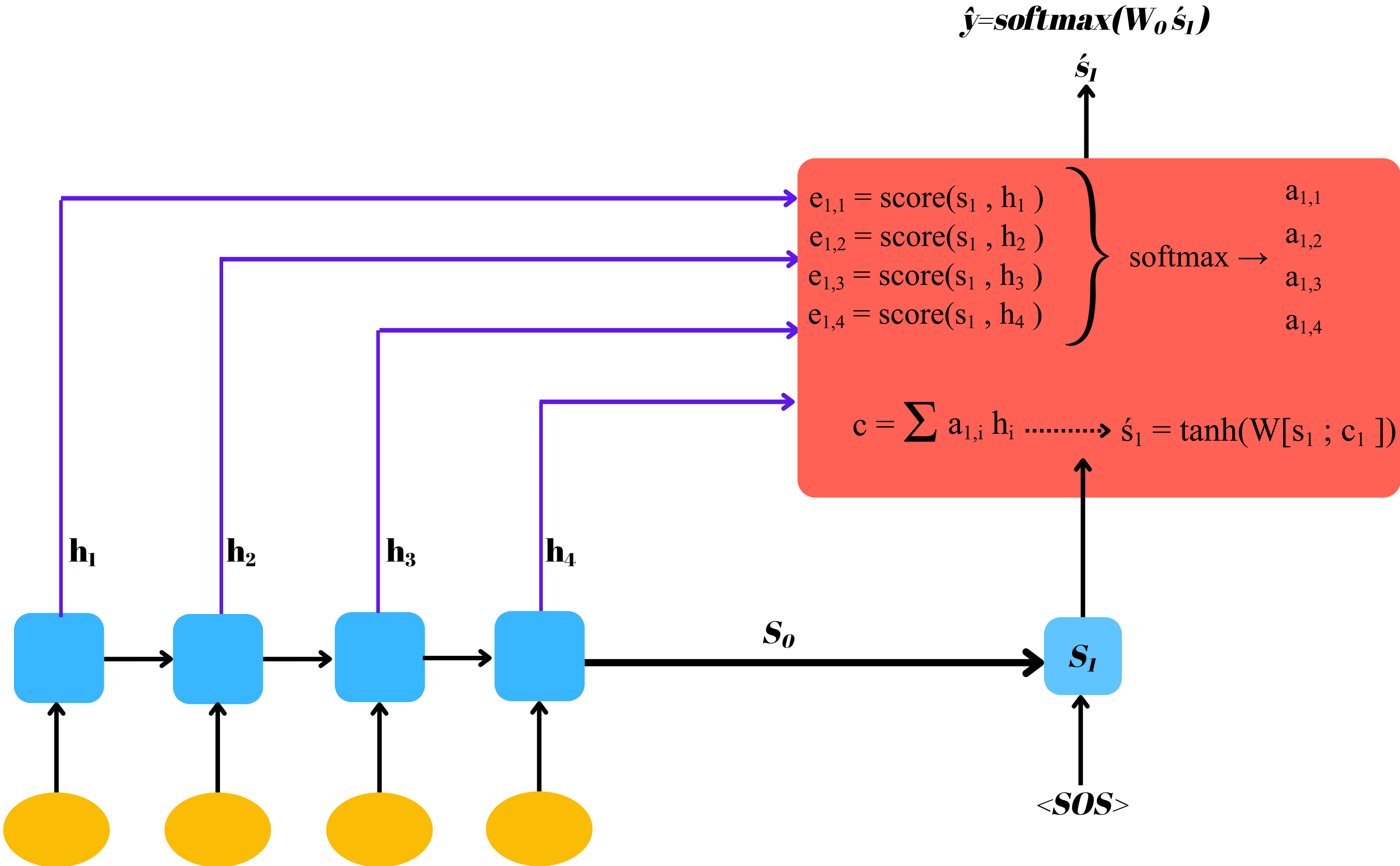
***The dog, which barked excessively at night and frightened the old woman, is now silent.***

- **Context Vector(s) uzun sequence boyunca anlamı yitirebilir. LSTM vb. bir miktar önüne geçse de yine de anlam arada değişime uğrayabilir.**
- **RNN-LSTM-GRU yapılarını decoder kısmının encoder kısmına sadece context vector(s) üzerinden bağlıdır. Attention bu sorunu çözmek için decoder kısmının encoder kısmındaki her bir hidden state'e erişimini sağlar.**

# <sup>1</sup>Attention Model

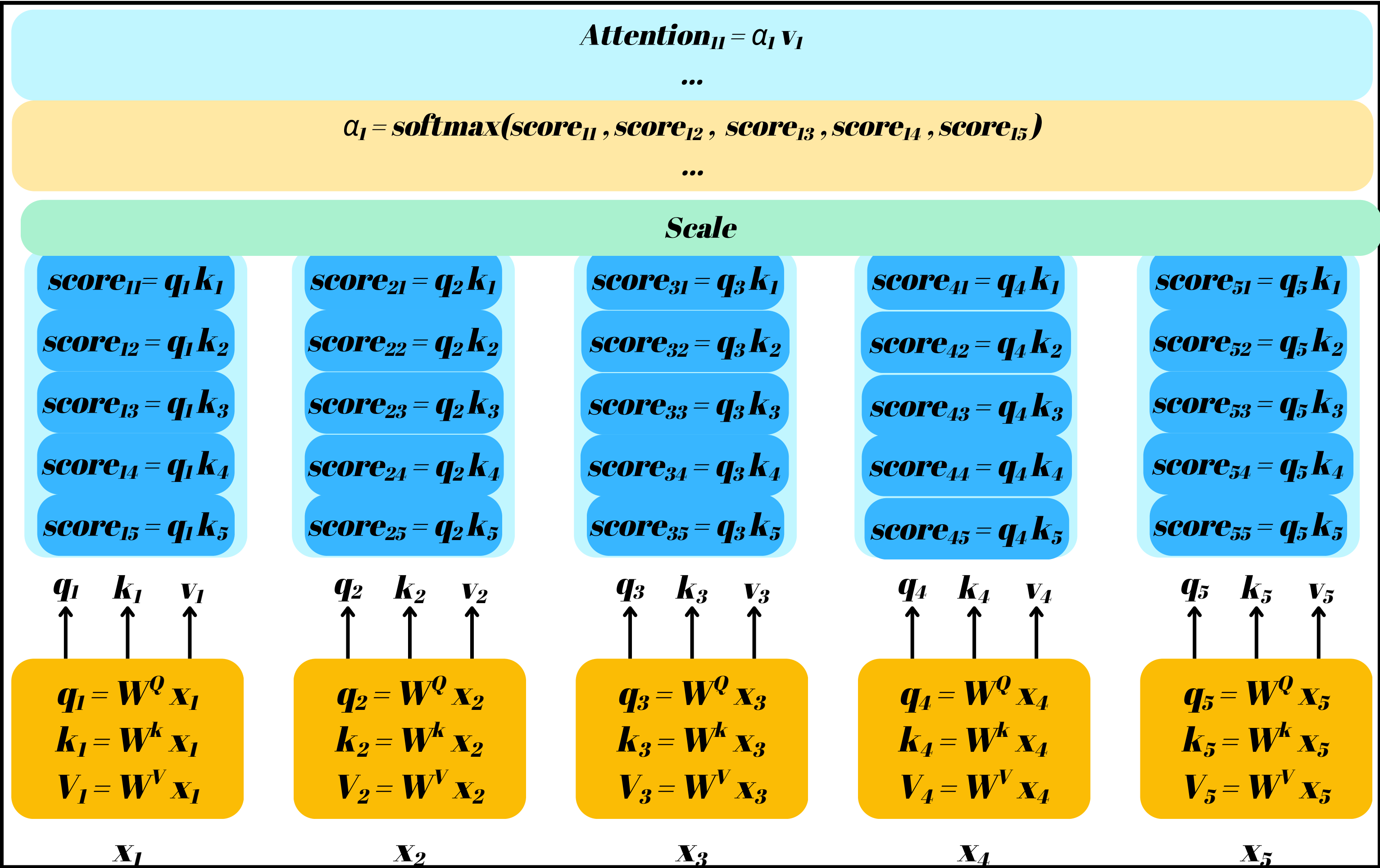


# 2Attention Model





# Self-Attention

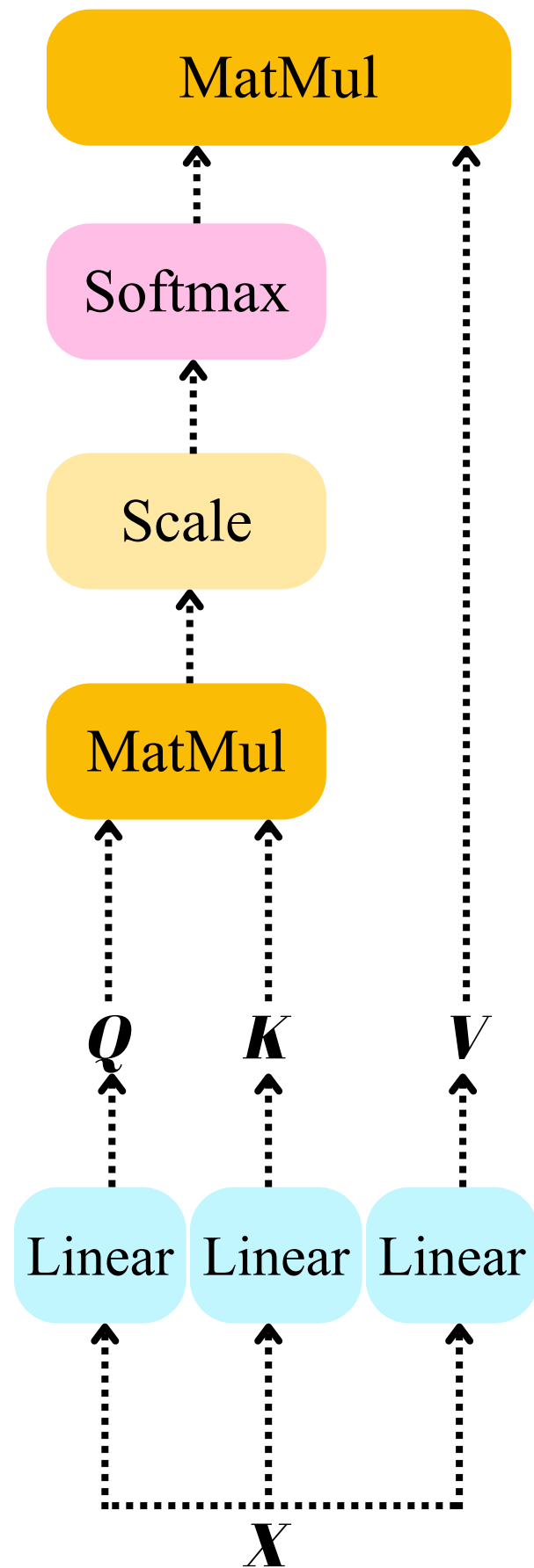


$q \rightarrow$  query  
 $k \rightarrow$  key  
 $v \rightarrow$  value

$W^q$   
 $W^k$   
 $W^v$  } ortak

**score**  
→ query'e uygun bir key mi  
→ Soruya uygun bir cevap mı

## <sup>2</sup>Self-Attention



$$Q = XW^Q \quad K = XW^K \quad V = XW^V$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

*Eğer input vektörler  $\rightarrow (300,1)$   
Sequence Length  $\rightarrow 5$*

$X \rightarrow (5,300)$   
 $W^Q \rightarrow (300,64)$   
 $W^K \rightarrow (300,64)$   
 $W^V \rightarrow (300,64)$

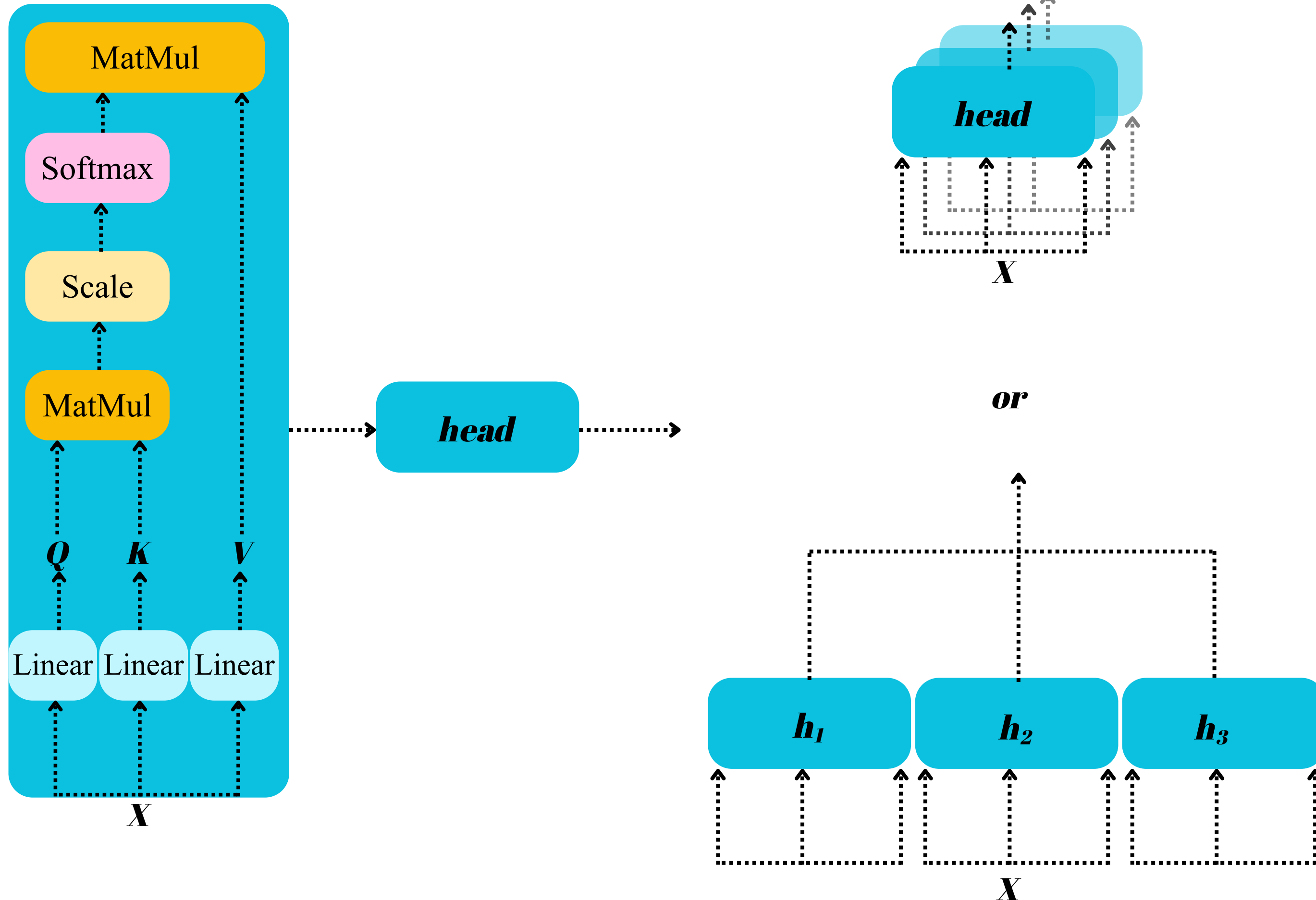
**Öğrenilen parametreler**

$Q \rightarrow (5,64)$   
 $K \rightarrow (5,64)$   
 $V \rightarrow (5,64)$   
 $S \rightarrow (5 \times 5)$   
 $Z = SV \rightarrow [5, 64]$   
**Output  $\rightarrow [5, 300]$**

*Herhangi bir token için word embedding girdi ve:*

- *Diğer tokenlar ile etkileşimi olan context-aware*
- *Semantic anlam taşıyan*
- *Position bilgisi taşıyan (Positional Encoding ile birlikte) yeni embeddingler çıktı*

# MultiHead Attention



- Head'ler paralelde hesaplama yapar ve bütün  $h$  çıktılarının “ortalaması alınır”/ “projeksiyonu alınır”
- Her bir head kendi içerisinde anlamlı bir soruya cevap oluşturabilir. (Nerede, ne zaman vb.)
- Paper'da  $h=8$

# Positional Encoding

- Self Attention doğrudan tokenların sıra bilgisini içermez. Positional encoding Word embedding üzerine eklenerek vektörlere sıra bilgisini de katar.

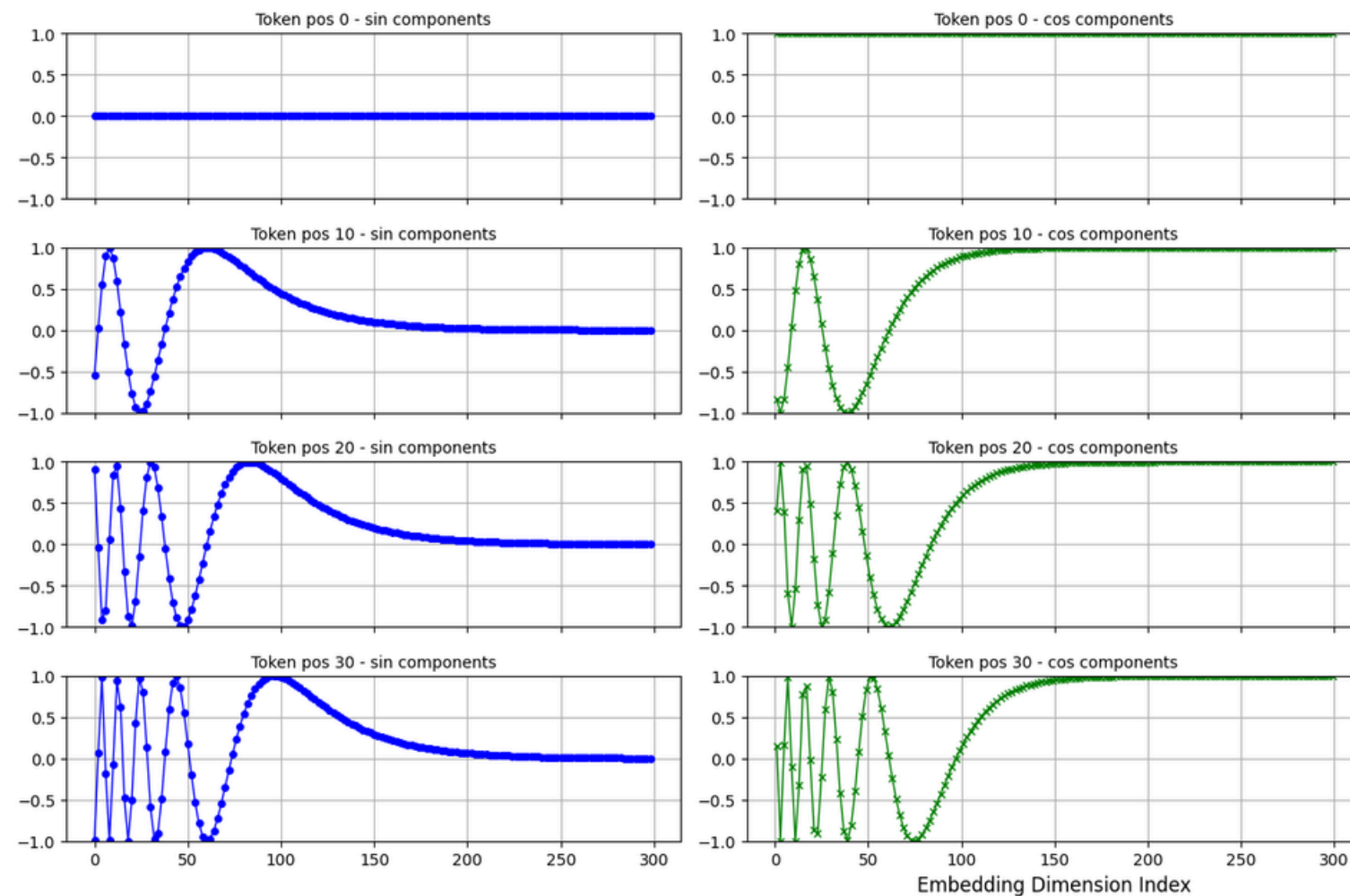
$max\_len = 5$

$d\_model = 8$

***Final Embedding = Word Embedding + Positional Embedding***

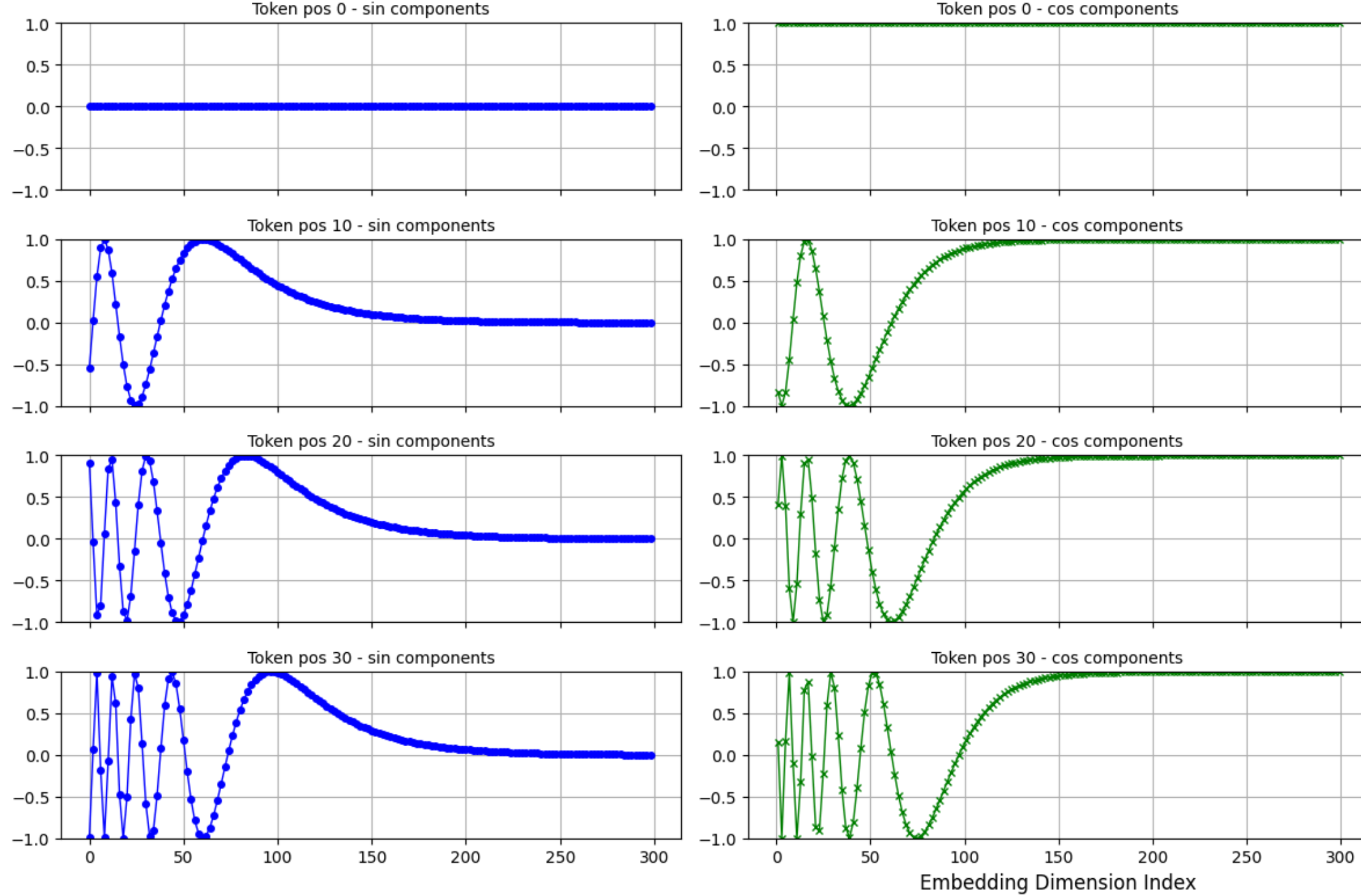
$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$



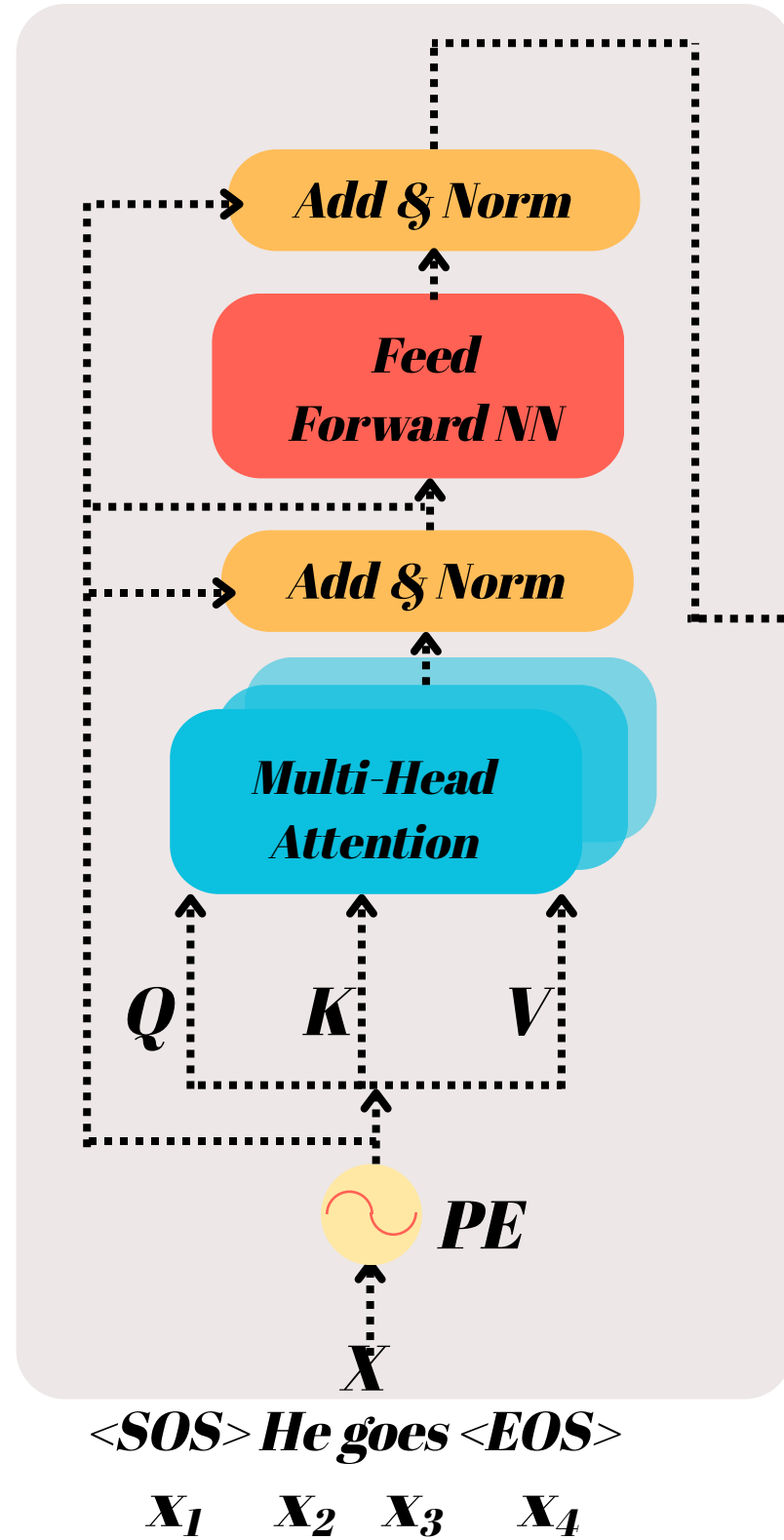
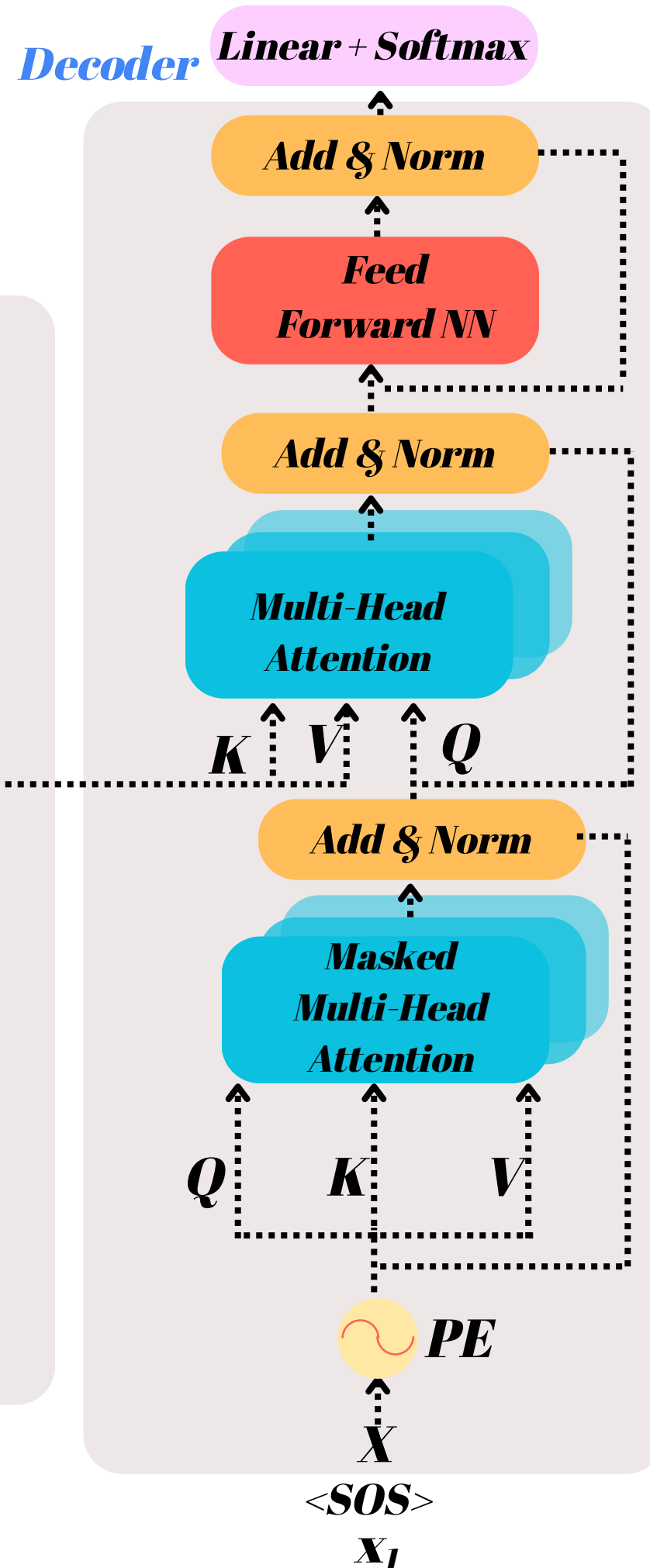
|         |         |        |        |        |        |   |   |
|---------|---------|--------|--------|--------|--------|---|---|
| 0       | 1       | 0      | 1      | 0      | 1      | 0 | 1 |
| 0.8415  | 0.5403  | 0.0084 | 0.9999 | 0.0001 | 1      | 0 | 1 |
| 0.9093  | -0.4161 | 0.0168 | 0.9999 | 0.0002 | 1      | 0 | 1 |
| 0.1411  | -0.9900 | 0.0252 | 0.9997 | 0.0003 | 0.9999 | 0 | 1 |
| -0.7568 | -0.6536 | 0.0336 | 0.9994 | 0.0003 | 0.9999 | 0 | 1 |

## <sup>2</sup>Positional Encoding



- **Positional Embedding için ilk dimension'daki girdiler çok hızlı değişir. Son dimension'lara doğru sabit gibi davranır.**
- **Eğer Word embedding boyutu küçükse Final embedding aşırı değişken olabilir.**
- **PE her bir girdi için özel sıra bilgisi oluşturur ve bunu embedding'in üzerine ekler.**
- **PE vektörleri NN modelleri tarafından da öğrenilebilir.**
- **PE için max\_token bilgisine ihtiyaç vardır. PE bilgileri bir tabloda tutulur tekrar tekrar hesaplanmaz. max\_token aşılsa look-up işlemi yapılamaz**



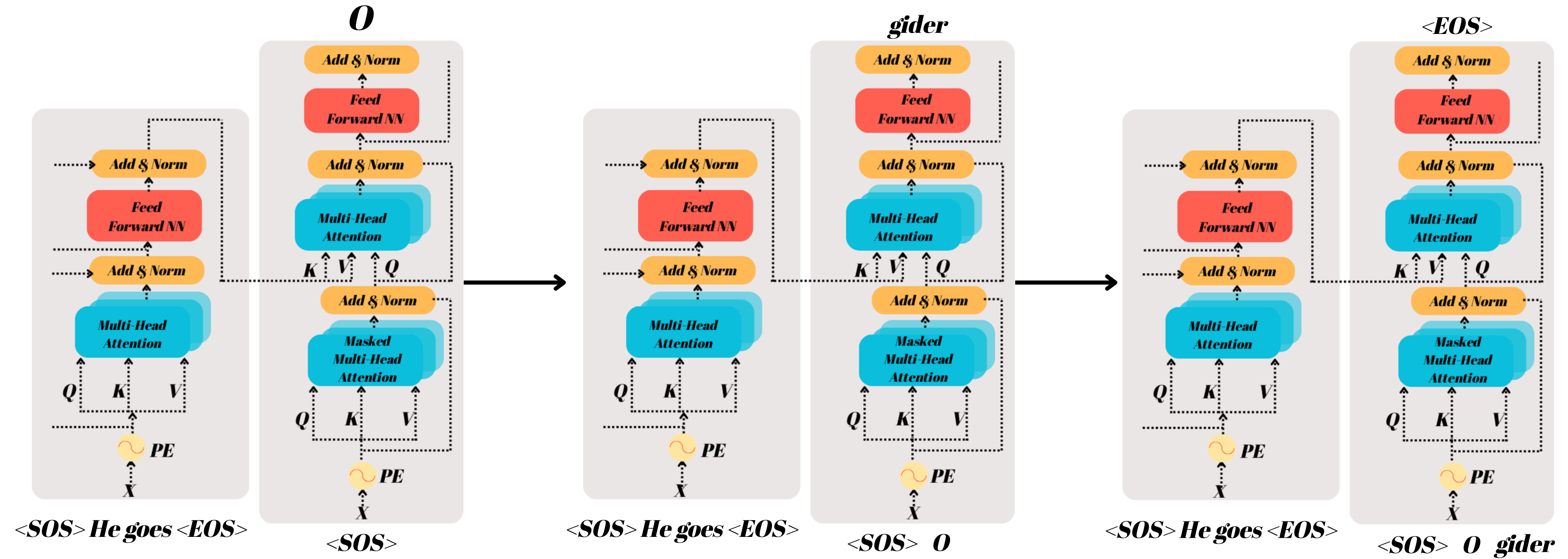
**Encoder****Decoder****Transformer**

- Encoder input sequence'ı , decoder output sequence işler. Encoder input'tan bilgi çıkartır ve bunu decoder kısmına aktarır, decoder o ana kadar verilen output'u ve encoder'dan gelen bilgiyi kullanarak çıktı verir.
- Decoder kısmındaki K,V,Q için O ana kadar verilen çıktıya uygun Q ile soru soruluyor ve bu sorunun cevabı encoderdan öğrenilen K,V içerisinde bulunuyor olarak benzetilebilir..
- Residual connection ile daha önceden öğrenilen bilginin kaybolmaması ve pozisyonel bilginin farklı kısımlar arasında paylaşılması sağlanıyor.
- Masked kısmı sadece Transformer eğitim kısmı için önemli olan bir kavramdır. (LLM kısımlarında değineceğiz muhtemelen)

*Residual  
Connection*



# Autoregressive Generation with Transformers



## İçeriği için

|                             |   |            |
|-----------------------------|---|------------|
| <i>Large Language Model</i> | } | <i>6.1</i> |
| <i>Foundation models</i>    |   | <i>6.2</i> |
| .                           | } | .          |
| .                           |   | .          |
| .                           |   | .          |

- **Stanford CS224N Natural Language Processing with Deep Learning:** Stanford Üniversitesi tarafından verilen dersin kaydının paylaşıldığı bir kurs. NLP alanı üzerine yoğunlaşan bir kurs ve şu ana kadar anlattığımız birçok şeye değiniyor. Aşırı detay AI bilgisi içermediği kanaatindeyim.

<https://www.youtube.com/playlist?list=PLoROMvody4rOaMFbaqxPDoLWjDaRAdP9D>

- **Andrew Ng tarafından verilen “Deep Learning” eğitiminin 5. kursu olan NLP kursu.** İçerisinde modül boyunca gördüğümüz birçok konu ve arkasındaki matematik detaylı olarak mevcut. NLP alanında özelleşmek isteyenler için temel kurslardan birisi sayılabilir.

<https://learn.deeplearning.ai/specializations/deep-learning/lesson/pk8f1/why-sequence-models%3F>